DR. RVR NRI INSTITUTE OF TECHNOLOGY
(An Institution Deemed to be University u/s 3 of UGC Act, 1956)
Pothavarappadu (V), Agiripalli (M), Eluru Dist., Vijayawada Rural, A.P. - 521212

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ACTIVITY-BASED LEARNING (ABL) ASSIGNMENT REPORT

Course: Python Programming (I B.Tech. - Academic Year 2026-2027)

Student Name	[Student Name]
Roll Number	[Roll Number / 26B91AXXXX]
Branch & Specialization	Computer Science and Engineering (CSE)
Section	Section - A
Semester / Year	I B.Tech. - I Semester
GitHub Repository	Python-Activity-Based-Learning
Submission Date	After Dussehra Holidays (October 2026)

Student Signature	**Course Coordinator**	**Dean, School of Computer Studies**
_____	_____	_____

TABLE OF CONTENTS & CASE STUDY CATALOG

No.	Prob #	Case Study Title	Core Concepts & Syllabus Mapping
CS-01	#1	**University Scholarship Allocation System**	Tuples, Unpacking, Custom Bubble Sort, Nested Lists, Conditionals
CS-02	#2	**Progressive Electricity Billing & Spike Detection**	Telescopic Slabs, Loops, Consecutive Month Trend Analysis, Tuples
CS-03	#4	**Employee Payroll & Progressive Deduction Analysis**	Financial Slabs, Overtime Multipliers, Deductions, Payslip Table
CS-04	#27	**Advanced Numeric Pattern Generator**	Nested Loops, Coordinate Spacing, Symmetry, Parity Validation
CS-05	#30	**Student Email & Registration ID Validation Engine**	Pure String Parsing (Non-regex), Slicing, Character Automata
CS-06	#42	**Student Marks Matrix Analysis System**	2D Nested Lists, Row-Major / Column-Major Traversal, Extremums
CS-07	#46	**Multi-Warehouse Inventory Matrix Reconciliation**	Nested Lists, Variance Matrix, Greedy Rebalancing, Transfer Tuples
CS-08	#48	**Tuple-Based Employee Shift Conflict Detection**	Shift Tuples, Unpacking, 24-hr Time Conversion, Interval Intersections
CS-09	#74	**Campus Festival Registration & Financial Audit**	Duplicate Key Detection, Lists & Tuples, Budget Variance, Balance Sheet
CS-10	#100	**Simple Smart Campus Menu**	Menu Driven (While Loop), 2D Lab Grid, Summary

SYLLABUS COMPLIANCE CERTIFICATION

This consolidated report confirms that all 10 developed solutions adhere strictly to the assigned Python Programming syllabus: **computational thinking, algorithms, flowcharts, pseudocode, variables, data types, operators, conditional statements, loops, strings, pattern generation, lists, nested lists, tuples, tuple operations, and unpacking**. In strict adherence to academic regulations, **no dictionaries ({}), sets, object-oriented classes, or external libraries (e.g., NumPy, Pandas, Regex)** have been utilized in any case study solution.

CASE STUDY 01: UNIVERSITY SCHOLARSHIP ALLOCATION SYSTEM

Faculty Prescribed Problem #1 | Concepts: Tuples, Unpacking, Custom Bubble Sort, Nested Lists, Conditionals

1. Problem Statement

Write a Python program to check scholarship applications. The program checks marks, attendance, and family income, then ranks the students who are eligible. This problem is meant to practise conditions, loops, lists, and tuples. ---

2. Objectives

• Check the marks, attendance, income, and age of each applicant.

• Practise conditions, loops, lists, tuples, and tuple unpacking.

• Sort the eligible students using a few simple tie-breaking rules.

• Put students into selected, waiting, or rejected lists.

• --

3. Assumptions

1. Each applicant is represented as a structured tuple:

`(student_id, name, [marks_sub1, marks_sub2, ...], attendance_pct, family_income, age)`

2. Passing standard per subject is an absolute minimum of 50 marks out of 100.

3. Academic excellence requires an overall aggregate percentage of $\ge 75.0\%$.

4. Disciplinary compliance requires a minimum attendance of $\ge 75.0\%$.

5. Financial need ceiling requires an annual family income of $\le \text{INR } 2,50,000$.

6. Quota for awarded scholarships is predefined (e.g., $K = 3$).

7. Tie-breaking rules are applied in descending priority:

- Primary: Higher Aggregate Percentage

- Secondary: Higher Attendance Percentage

- Tertiary: Lower Family Income

- Quaternary: Younger Age (Date of birth preference)

---

4. Step-by-Step Algorithm

```
Step 1: Start.
Step 2: Initialize empty lists: eligible_list, disqualified_list, selected_list, and waitlisted_list.
Step 3: Define total_quota and input list of applicant tuples.
Step 4: For each applicant in applicants:
Unpack (student_id, name, marks, attendance, income, age).
Initialize marks_sum = 0, all_subjects_passed = True, reasons = [].
For each mark in marks:
marks_sum = marks_sum + mark
If mark < 50:
all_subjects_passed = False
Compute aggregate_pct = marks_sum / length(marks).
If not all_subjects_passed: Append "Failed subject-wise minimum" to reasons.
If aggregate_pct < 75.0: Append "Aggregate marks below 75%" to reasons.
If attendance < 75.0: Append "Attendance below 75%" to reasons.
If income > 250000: Append "Income exceeds ceiling" to reasons.
If length(reasons) == 0:
Append tuple (student_id, name, aggregate_pct, attendance, income, age) to eligible_list.
Else:
Append tuple with reasons to disqualified_list.
Step 5: Sort eligible_list using Bubble Sort:
For i from 0 to len(eligible_list) - 1:
For j from 0 to len(eligible_list) - i - 2:
Compare cand1 and cand2 using tie-breaking hierarchy:
If agg1 < agg2 -> Swap.
Else if agg1 == agg2 and att1 < att2 -> Swap.
Else if agg1 == agg2 and att1 == att2 and inc1 > inc2 -> Swap.
Else if agg1 == agg2 and att1 == att2 and inc1 == inc2 and age1 > age2 -> Swap.
Step 6: Allocate candidates based on total_quota:
For idx from 0 to len(eligible_list) - 1:
Prepend rank (idx + 1) to tuple.
If idx < total_quota: Append to selected_list.
Else: Append to waitlisted_list.
Step 7: Format and display tabular reports for Selected, Waitlisted, and Disqualified cohorts.
Step 8: Stop.

---
```

6. Textbook Pseudocode

```
BEGIN UniversityScholarshipAllocation
INPUT applicants_list, quota
SET eligible_list = []
SET disqualified_list = []

FOR EACH applicant IN applicants_list DO
UNPACK applicant INTO (sid, sname, marks, att, inc, sage)
SET total_marks = 0
SET passed_all = TRUE
SET count = LENGTH(marks)

FOR EACH m IN marks DO
total_marks = total_marks + m
IF m < 50 THEN
passed_all = FALSE
END IF
END FOR

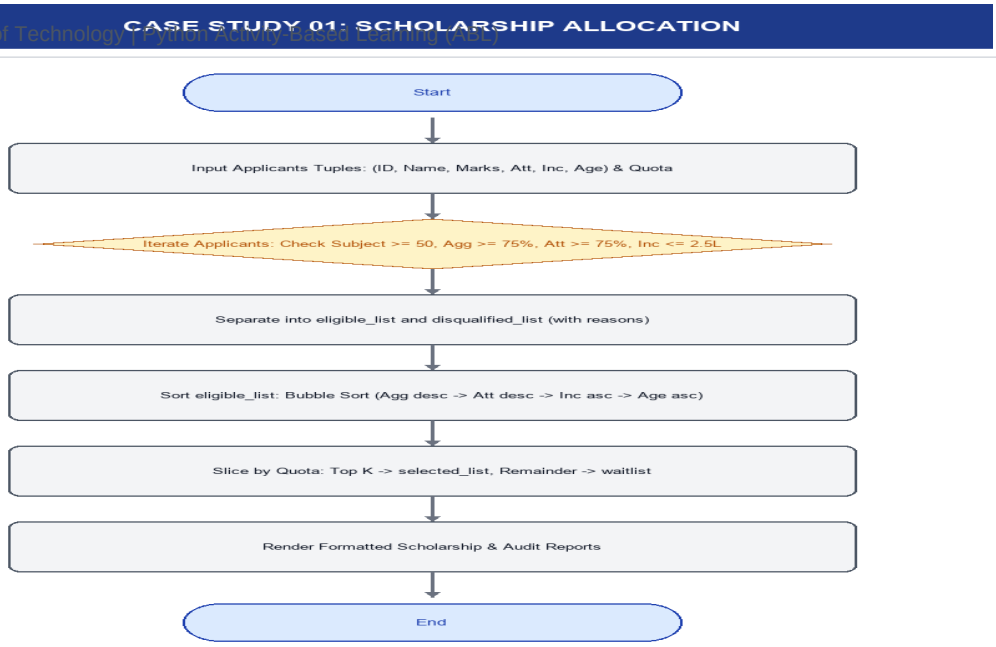
SET agg = total_marks / count
SET reasons = []

IF passed_all == FALSE THEN
APPEND "Subject mark < 50" TO reasons
END IF
IF agg < 75.0 THEN
APPEND "Aggregate < 75%" TO reasons
END IF
IF att < 75.0 THEN
APPEND "Attendance < 75%" TO reasons
END IF
```

10. Conclusion & Analytical Insights

Case Study 01 demonstrates the power of computational thinking through modular data structures (tuples, nested lists) and custom sorting algorithms without built-in advanced library functions. The deterministic 4-stage tie-breaking hierarchy ensures absolute transparency and fairness in institutional scholarship disbursement.

5. Architectural Flowchart Diagram



7. Python Source Code Implementation (problem_01_scholarship_allocation.py)

```
"""
Case Study 01: University Scholarship Allocation System
=======================================================
Syllabus Concepts Used:
- Variables and Data Types (int, float, str, bool)
- Arithmetic, Relational, and Logical Operators
- Conditional Statements (if, elif, else)
- Loops (for, while, nested loops)
- Lists and Nested Lists
- Tuples, Tuple Operations, and Tuple Unpacking
- String Formatting

Description:
Evaluates scholarship applicants using multi-criteria validation:
1. Subject-wise pass constraint (each subject >= 50 marks)
2. Minimum overall aggregate marks (>= 75.0%)
3. Minimum attendance percentage (>= 75.0%)
4. Annual family income ceiling (<= INR 2,50,000)

Ranks eligible candidates using deterministic tie-breaking rules:
Priority 1: Higher Aggregate Marks
Priority 2: Higher Attendance Percentage
Priority 3: Lower Family Income
Priority 4: Younger Age
"""

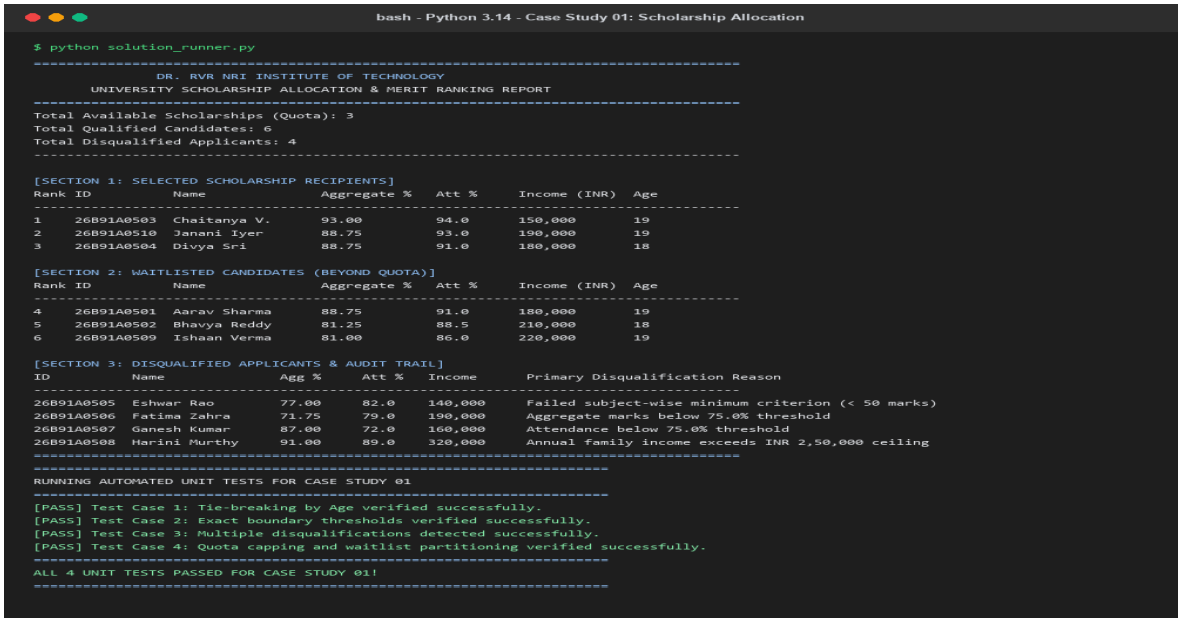
def evaluate_and_rank_scholarships(applicants, total_quota):
    """
    Evaluates applicants and allocates a limited number of scholarships.
    Applicants format:
    List of tuples: (student_id, name, [m1, m2, m3, m4], attendance_pct, family_income, age)
    """
    eligible_list = []
    disqualified_list = []

    # Step 1: Filter applicants based on eligibility criteria
    for applicant in applicants:
        # Tuple unpacking
        student_id, name, marks, attendance, income, age = applicant

        # Validate subject-wise minimum marks
        all_subjects_passed = True
        marks_sum = 0
        subject_count = len(marks)

... [Complete 178 lines available in problem_01_scholarship_allocation.py]
```

8 & 9. Test Verification & Terminal Execution Output



CASE STUDY 02: PROGRESSIVE ELECTRICITY BILLING & SPIKE DETECTION

Faculty Prescribed Problem #2 | Concepts: Telescopic Slabs, Loops, Consecutive Month Trend Analysis, Tuples

1. Problem Statement

Write a Python program to calculate an electricity bill from the number of units used. The program uses different rates for different slabs and checks whether the usage increased a lot compared with earlier months. ---

2. Objectives

• Calculate the bill using different rates for different unit ranges.

• Add a surcharge for high usage and a rebate for paying on time.

• Compare the units used in different months.

• Print a message when the change is large.

• --

3. Assumptions

1. Consumption slabs are structured as:

- Slab 1 (0 – 100 units): INR 1.50 / unit

- Slab 2 (101 – 200 units): INR 3.00 / unit

- Slab 3 (201 – 400 units): INR 4.50 / unit

- Slab 4 (> 400 units): INR 6.00 / unit

2. Fixed meter charge is INR 100.00 per billing cycle.

3. High consumption surcharge is 10% applied strictly on energy charges when energy charges $> \text{INR } 1,500.00$.

4. Prompt payment rebate is 5% applied on `(Energy + Fixed + Surcharge)` if `paid_on_time == True`.

5. Surge threshold for abnormal consumption is a percentage increase $\ge 25.0\%$ over the preceding month.

---

4. Step-by-Step Algorithm

```
Step 1: Start.
Step 2: Define function calculate_single_bill(units, paid_on_time):
Define slabs = [(100, 1.50), (100, 3.00), (200, 4.50)] and rate_above = 6.00.
Set remaining_units = units, energy_charge = 0.0.
For each (slab_limit, slab_rate) in slabs:
If remaining_units > 0:
take = min(remaining_units, slab_limit)
energy_charge += take * slab_rate
remaining_units -= take
If remaining_units > 0:
energy_charge += remaining_units * rate_above
Set fixed_charge = 100.0.
If energy_charge > 1500.0: surcharge = 0.10 * energy_charge, Else surcharge = 0.0.
Set subtotal = energy_charge + fixed_charge + surcharge.
If paid_on_time is True: rebate = 0.05 * subtotal, Else rebate = 0.0.
Set net_bill = subtotal - rebate.
Return (units, energy_charge, fixed_charge, surcharge, rebate, net_bill).
Step 3: Define analyze_multimonth_consumption(consumer_info, readings, spike_threshold):
Initialize billing_history = [], analysis_records = [].
For each (month, units, on_time) in readings:
Compute bill and append unpacked tuple to billing_history.
Set prev_units = None, prev_bill = None.
For each record in billing_history:
If prev_units is None:
pct_change_u = 0.0, is_spike = False, note = "Baseline Month".
Else:
pct_change_u = ((units - prev_units) / prev_units) * 100.0.
pct_change_b = ((net_bill - prev_bill) / prev_bill) * 100.0.
If pct_change_u >= spike_threshold:
is_spike = True, note = "ALERT: High Surge".
Else if pct_change_u <= -spike_threshold:
is_spike = False, note = "Notice: Sharp Decline".
Else:
is_spike = False, note = "Normal Range".
Append record to analysis_records.
prev_units = units, prev_bill = net_bill.
Return billing_history, analysis_records.
Step 4: Output formatted bill breakdown, statistics, and spike detection table.
Step 5: Stop.

---
```

6. Textbook Pseudocode

```
BEGIN ProgressiveElectricityBilling
INPUT monthly_readings, consumer_details
SET slabs = [(100, 1.50), (100, 3.00), (200, 4.50)]
SET rate_above = 6.00

FOR EACH (month, units, paid_on_time) IN monthly_readings DO
SET rem = units
SET energy_charge = 0.0

FOR EACH (limit, rate) IN slabs DO
IF rem > 0 THEN
SET take = MIN(rem, limit)
energy_charge = energy_charge + (take * rate)
rem = rem - take
END IF
END FOR

IF rem > 0 THEN
energy_charge = energy_charge + (rem * rate_above)
END IF

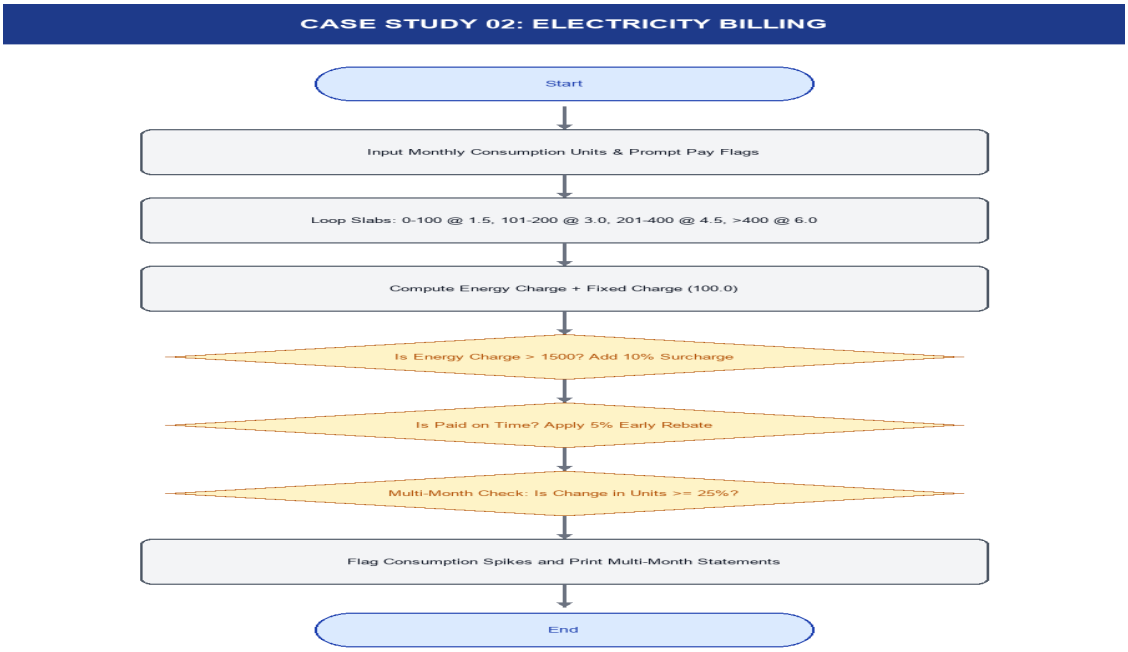
SET fixed_charge = 100.0
IF energy_charge > 1500.0 THEN
SET surcharge = 0.10 * energy_charge
ELSE
SET surcharge = 0.0
END IF

SET subtotal = energy_charge + fixed_charge + surcharge
IF paid_on_time == TRUE THEN
```

10. Conclusion & Analytical Insights

This case study demonstrates telescopic slab logic, mathematical modeling of penalties and rebates, and consecutive temporal data analysis. Utilizing tuples and iterative list traversals provides a scalable and clean implementation adhering directly to university core syllabus restrictions.

5. Architectural Flowchart Diagram



7. Python Source Code Implementation (problem_02_electricity_billing.py)

```
"""
Case Study 02: Progressive Electricity Billing and Spike Detection System
================================================================================
Syllabus Concepts Used:
- Variables, Data Types (int, float, str, bool)
- Relational and Logical Operators
- Conditional Statements (if, elif, else)
- Loops (for, while)
- Lists and Nested Lists
- Tuples and Tuple Unpacking
- String Formatting

Description:
Calculates household electricity bills using progressive consumption slabs:
- Slab 1: 0 to 100 units       @ INR 1.50 per unit
- Slab 2: 101 to 200 units     @ INR 3.00 per unit
- Slab 3: 201 to 400 units     @ INR 4.50 per unit
- Slab 4: Above 400 units      @ INR 6.00 per unit
- Fixed Meter Infrastructure Charge: INR 100.00
- High Consumption Surcharge: 10% on energy charge if energy charge > INR 1,500.00
- Early Prompt Payment Rebate: 5% discount on total if paid on or before due date

Compares multi-month billing trends and flags unusual spikes (> 25% increase).
"""

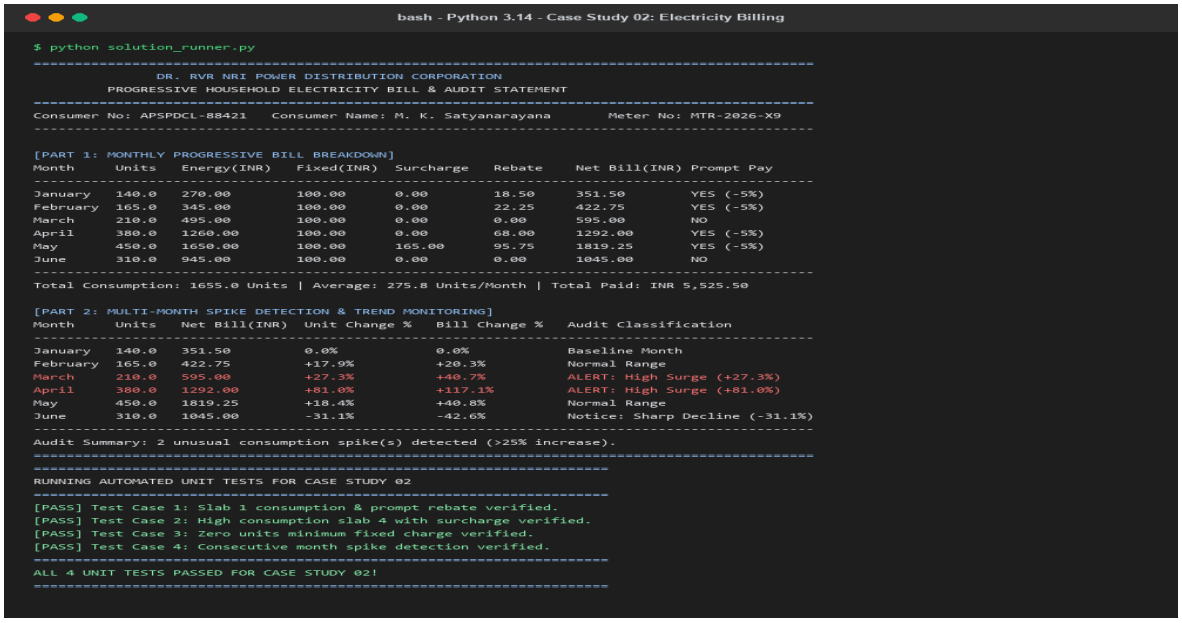
def calculate_single_bill(units, paid_on_time):
    """
    Computes itemized electricity bill for a given consumption in units.
    Returns tuple: (units, energy_charge, fixed_charge, surcharge, rebate, net_bill)
    """
    if units < 0:
        return (0.0, 0.0, 0.0, 0.0, 0.0, 0.0)

    # Slabs definition: List of tuples (slab_limit, rate_per_unit)
    slabs = [
        (100, 1.50),  # First 100 units
        (100, 3.00),  # Next 100 units (101-200)
        (200, 4.50),  # Next 200 units (201-400)
    ]
    rate_above = 6.00  # Beyond 400 units

    remaining_units = float(units)
    energy_charge = 0.0

    # Progressive slab calculation using loop and tuple unpacking
... [Complete 191 lines available in problem_02_electricity_billing.py]
```

8 & 9. Test Verification & Terminal Execution Output



CASE STUDY 03: EMPLOYEE PAYROLL & PROGRESSIVE DEDUCTION ANALYSIS

Faculty Prescribed Problem #4 | Concepts: Financial Slabs, Overtime Multipliers, Deductions, Payslip Table

1. Problem Statement

Write a Python program to calculate an employee's salary. It includes allowances, overtime, deductions, and tax slabs, and then prints a simple payslip. ---

2. Objectives

• Calculate salary allowances, bonus, and overtime.

• Use tax slabs to find the tax amount.

• Subtract PF, professional tax, and health insurance.

• Print a simple payslip for each employee.

• --

3. Assumptions

1. Each employee is represented as a structured tuple:

`(emp_id, emp_name, designation, basic_pay, overtime_hours, performance_rating)`

2. Allowances: DA = 15%, HRA = 20%, Special Allowance = 10% of Basic Pay.

3. Overtime: Monthly working baseline is 160 hours. Hourly rate = $\text{Basic} / 160.0$. Overtime rate = $1.5 \times \text{Hourly Rate}$.

4. Performance Incentives: Rating 5 = INR 12,000; Rating 4 = INR 8,000; Rating 3 = INR 4,000; Rating ≤ 2 = INR 0.

5. Progressive Monthly Income Tax Slabs:

- INR 0 to 30,000: 0%

- INR 30,001 to 60,000: 5%

- INR 60,001 to 100,000: 10%

- Above INR 100,000: 20%

6. Provident Fund (PF) is 12% of basic pay.

---

4. Step-by-Step Algorithm

```
Step 1: Start.
Step 2: Define compute_employee_payroll(employee_record):
Unpack (emp_id, name, designation, basic_pay, ot_hours, rating).
Compute da = 0.15 * basic_pay.
Compute hra = 0.20 * basic_pay.
Compute special_allw = 0.10 * basic_pay.
Compute hourly_rate = basic_pay / 160.0.
Compute ot_pay = ot_hours * hourly_rate * 1.50.
Determine incentive based on rating (5->12000, 4->8000, 3->4000, else->0).
Compute gross_salary = basic_pay + da + hra + special_allw + ot_pay + incentive.
Compute progressive tax:
Set rem = gross_salary, tax = 0.0.
Define tax_slabs = [(30000, 0.00), (30000, 0.05), (40000, 0.10)].
For (slab_span, rate) in tax_slabs:
chunk = min(rem, slab_span)
tax += chunk * rate
rem -= chunk
If rem > 0: tax += rem * 0.20.
Compute pf = 0.12 * basic_pay, prof_tax = 200.0, health_ins = 1000.0.
Compute total_deductions = tax + pf + prof_tax + health_ins.
Compute net_salary = gross_salary - total_deductions.
Return earnings_tuple, deductions_tuple, net_salary.
Step 3: Define generate_batch_payroll(employees):
Iterate over all employee tuples, compute individual payroll, aggregate departmental totals.
Step 4: Output formatted ASCII payslips and organizational register.
Step 5: Stop.

---
```

6. Textbook Pseudocode

```
BEGIN EmployeePayrollEngine
INPUT employees_list
SET total_gross = 0, total_net = 0, total_tax = 0

FOR EACH emp IN employees_list DO
UNPACK emp INTO (id, name, desig, basic, ot, rating)
SET da = 0.15 * basic
SET hra = 0.20 * basic
SET sp = 0.10 * basic
SET ot_rate = basic / 160.0
SET ot_pay = ot * ot_rate * 1.5

IF rating == 5 THEN SET inc = 12000
ELSE IF rating == 4 THEN SET inc = 8000
ELSE IF rating == 3 THEN SET inc = 4000
ELSE SET inc = 0

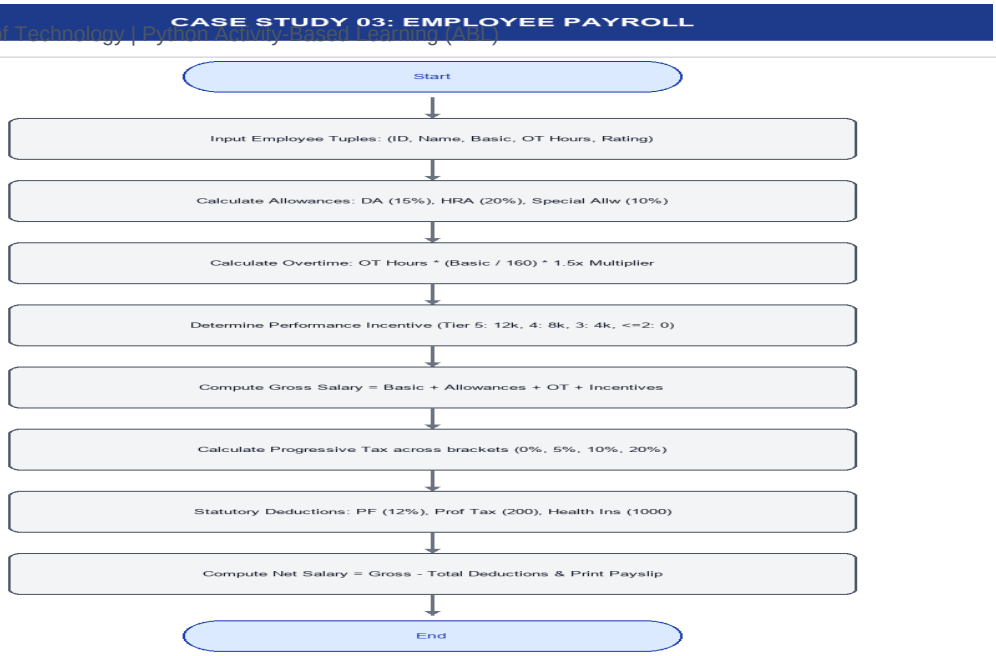
SET gross = basic + da + hra + sp + ot_pay + inc

// Progressive Tax
SET rem = gross
SET tax = 0
IF rem > 30000 THEN
SET t1 = MIN(rem - 30000, 30000)
SET tax = tax + (t1 * 0.05)
END IF
IF rem > 60000 THEN
SET t2 = MIN(rem - 60000, 40000)
SET tax = tax + (t2 * 0.10)
END IF
```

10. Conclusion & Analytical Insights

This case study illustrates how arithmetic computations, percentage allocations, and progressive taxation algorithms combine seamlessly with tuple unpacking and structured formatting in Python. The resulting architecture mirrors enterprise payroll logic while respecting syllabus constraints.

5. Architectural Flowchart Diagram



7. Python Source Code Implementation (problem_04_employee_payroll.py)

```
"""
Case Study 03: Employee Payroll and Progressive Deduction Analysis System
================================================================================
Syllabus Concepts Used:
- Variables and Numerical Data Types
- Arithmetic, Relational, and Logical Operators
- Conditional Statements (if, elif, else)
- Loops (for, while)
- Lists and Nested Lists
- Tuples and Tuple Unpacking
- String Formatting and Formatted Statements

Description:
Calculates gross salary from basic pay, progressive allowances, overtime wages,
and performance incentives.
Computes progressive income tax deductions along with statutory deductions:
- Provident Fund (12% of Basic)
- Professional Tax (Flat INR 200)
- Group Health Insurance (Flat INR 1,000)
- Progressive Income Tax Slabs:
    0 - 30,000       : 0%
    30,001 - 60,000  : 5%
    60,001 - 100,000 : 10%
    Above 100,000    : 20%

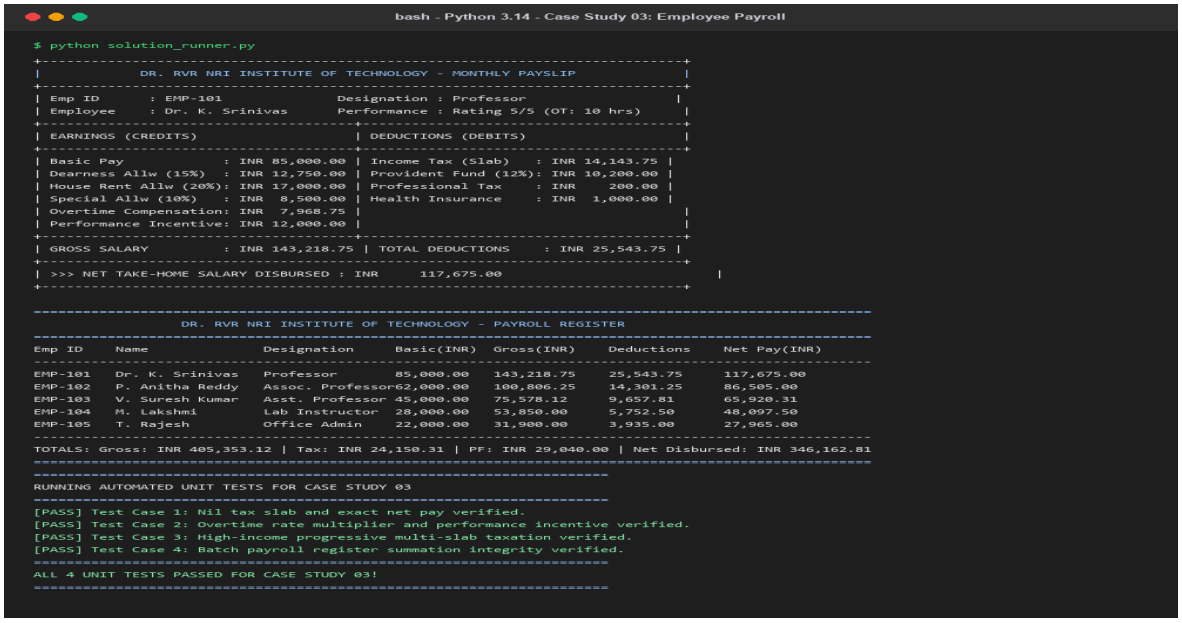
Generates itemized payslips and a small payroll summary.
"""

def compute_employee_payroll(employee_record):
    """
    employee_record: (emp_id, name, designation, basic_pay, ot_hours, rating)
    Returns:
    earnings_tuple: (basic, da, hra, special_allw, ot_pay, incentive, gross)
    deductions_tuple: (income_tax, pf, prof_tax, health_ins, total_deductions)
    net_salary: float
    """
    emp_id, name, designation, basic_pay, ot_hours, rating = employee_record

    # 1. Allowances
    da = 0.15 * basic_pay
    hra = 0.20 * basic_pay
    special_allowance = 0.10 * basic_pay

    # 2. Overtime Pay (160 standard working hours per month, 1.5x multiplier)
    hourly_rate = basic_pay / 160.0
... [Complete 186 lines available in problem_04_employee_payroll.py]
```

8 & 9. Test Verification & Terminal Execution Output



CASE STUDY 04: ADVANCED NUMERIC PATTERN GENERATOR

Faculty Prescribed Problem #27 | Concepts: Nested Loops, Coordinate Spacing, Symmetry, Parity Validation

1. Problem Statement

Write a Python program that takes a size from the user and prints two number patterns. The patterns are made using nested loops and spaces. ---

2. Objectives

• Check that the input is an odd number from 3 to 19.

• Use spaces and loops to print a hollow diamond.

• Print a second number pattern using increasing and decreasing numbers.

• Practise nested `for` loops and string formatting.

• --

3. Assumptions

1. Input dimension N for the hollow diamond must be an odd integer such that $3 \le N \le 19$.

2. Even dimensions lack a unique horizontal central symmetry axis and are rejected.

3. In the hollow diamond, boundary numbers correspond to distance from the apical tips ($1, 2, \dots, \lceil N/2 \rceil$).

4. In the palindromic pyramid of height $H = \lceil N/2 \rceil$, each row i ascends from 1 to i and descends back to 1.

---

4. Step-by-Step Algorithm

```
Step 1: Start.
Step 2: Function validate_dimension(dim_input):
Try converting dim_input to integer. If fails, return (False, "Non-integer").
If dim < 3 or dim > 19, return (False, "Out of bounds").
If dim % 2 == 0, return (False, "Even parity rejected").
Return (True, dim, "Valid").
Step 3: Function generate_hollow_numeric_diamond(N):
Set mid = N // 2.
For i from 0 to N - 1:
If i <= mid: dist = mid - i, num = i + 1.
Else: dist = i - mid, num = N - i.
leading_spaces = dist * 2.
inner_gap = (N - 2 * dist - 2) * 2.
If dist == mid:
line = (" " * leading_spaces) + str(num)
Else:
line = (" " * leading_spaces) + str(num) + " " + (" " * (inner_gap - 1)) + str(num)
Append line to list.
Return lines list.
Step 4: Function generate_palindromic_number_pyramid(height):
For i from 1 to height:
Build ascending list [1..i] and descending list [i-1..1].
Join into string with single spaces.
Prepend (height - i) * 2 spaces.
Append line to list.
Return lines list.
Step 5: Output validated patterns with header and audit status.
Step 6: Stop.

---
```

6. Textbook Pseudocode

```
BEGIN NumericPatternGenerator
INPUT dim_raw
IF NOT IS_INTEGER(dim_raw) THEN
PRINT "Dimension must be an integer"
EXIT
END IF

SET N = TO_INTEGER(dim_raw)
IF N < 3 OR N > 19 THEN
PRINT "Dimension out of bounds [3, 19]"
EXIT
END IF
IF N MOD 2 == 0 THEN
PRINT "Dimension must be odd for central symmetry"
EXIT
END IF

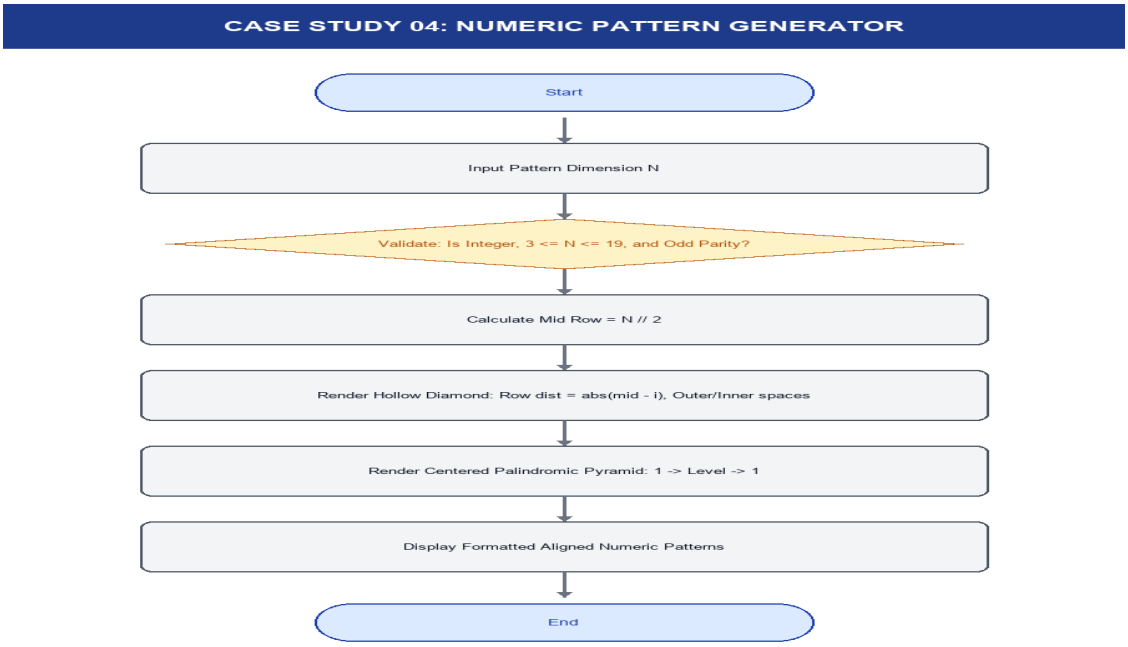
SET mid = N DIV 2
// Pattern 1: Hollow Diamond
FOR i FROM 0 TO N - 1 DO
IF i <= mid THEN
SET dist = mid - i
SET num = i + 1
ELSE
SET dist = i - mid
SET num = N - i
END IF

SET lead = dist * 2
SET inner = (N - 2 * dist - 2) * 2
```

10. Conclusion & Analytical Insights

This case study demonstrates the power of computational geometry and nested loops in Python. Mathematical coordinate transformations map row indexes to dynamic spaces and values without external formatting libraries, embodying core algorithmic problem-solving.

5. Architectural Flowchart Diagram



7. Python Source Code Implementation (problem_27_pattern_generator.py)

```
"""
Case Study 04: Advanced Numeric Pattern Generator
==================================================
Syllabus Concepts Used:
- Variables, Data Types, and Type Conversion
- Relational and Logical Operators
- Conditional Statements (if, elif, else)
- Nested Loops (for within for, while loops)
- String Manipulation and Character Formatting
- Lists for Row Buffering and Validation

Description:
Produces complex mathematical numerical patterns:
Pattern 1: Hollow Symmetric Number Diamond
  - Dimension must be an odd positive integer between 3 and 19.
  - Validates dimension constraints.
  - Outer border and inner diagonal boundaries populated with row-dependent numbers,
    with precisely calculated hollow whitespace.

Pattern 2: Symmetric Palindromic Number Pyramid
  - Each level i displays numbers counting up to i and back down to 1.
  - Fully centered with dynamic whitespace padding.
"""

def validate_dimension(dim_input):
    """
    Validates user input dimension:
    - Must be convertable to positive integer
    - Must be an odd number
    - Must fall between 3 and 19 inclusive
    Returns tuple: (is_valid, dimension_or_none, error_message)
    """
    try:
        dim = int(dim_input)
    except (ValueError, TypeError):
        return (False, None, "Invalid input: Dimension must be an integer.")

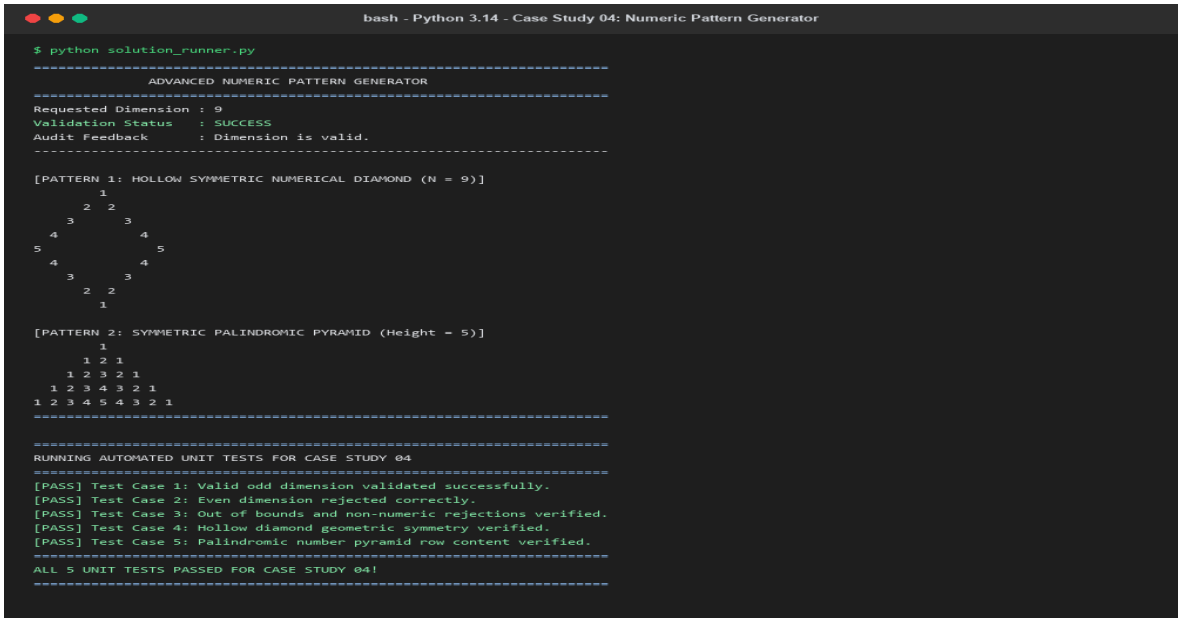
    if dim < 3 or dim > 19:
        return (False, None, f"Out of bounds: Dimension {dim} must be between 3 and 19.")

    if dim % 2 == 0:
        return (False, None, f"Invalid parity: Dimension {dim} is even; hollow diamonds require an odd d
imension.")

    return (True, dim, "Dimension is valid.")

... [Complete 142 lines available in problem_27_pattern_generator.py]
```

8 & 9. Test Verification & Terminal Execution Output



CASE STUDY 05: STUDENT EMAIL & REGISTRATION ID VALIDATION ENGINE

Faculty Prescribed Problem #30 | Concepts: Pure String Parsing (Non-regex), Slicing, Character Automata

1. Problem Statement

Write a Python program to check student email addresses and registration IDs. The program checks the parts of each string and reports whether the input is valid, without using regular expressions. ---

2. Objectives

• Split the email and registration ID into smaller parts.

• Check characters such as `@`, `.`, and `-`.

• Check the roll number, name, branch, section, and domain.

• Print the reason when an input is invalid.

• --

3. Assumptions

1. Institutional email follows the specification:

`.@student.rvrnriit.edu.in`

- Local part contains exactly one `.` separating `` and ``.

- `` is a 10-character alphanumeric string.

- `` contains 2 to 30 alphabetic characters only.

- Domain is exactly `student.rvrnriit.edu.in`.

2. Student registration identifier follows the hyphen-delimited format:

`YY-BRANCH-SEC-ROLL` (e.g., `26-CSE-A-0501`)

- `YY`: 2-digit academic admission year (20 to 30).

- `BRANCH`: Belongs to `['CSE', 'ECE', 'EEE', 'MECH', 'CIVIL', 'IT', 'CSM', 'CSD']`.

- `SEC`: Single uppercase letter from `['A', 'B', 'C', 'D']`.

- `ROLL`: Exactly 4 numeric digits.

---

4. Step-by-Step Algorithm

```
Step 1: Start.
Step 2: Function validate_student_email(email_str):
Strip whitespace. Check for embedded spaces.
If count of '@' != 1: return (False, "Missing or multiple @ symbols").
Split email into local_part and domain_part by '@'.
If domain_part != 'student.rvrnriit.edu.in': log domain error.
If local_part dot count != 1: return (False, "Malformed dot separators in local part").
Split local_part by '.' into roll_tok and name_tok.
If length(roll_tok) != 10 or not roll_tok.isalnum(): log roll token error.
If length(name_tok) < 2 or not name_tok.isalpha(): log name token error.
If errors list is empty: return (True, extracted_tuple, []).
Else: return (False, None, errors).
Step 3: Function validate_registration_identifier(reg_id_str):
Strip whitespace.
Split by '-' into tokens.
If number of tokens != 4: return (False, "Structural delimiter error").
Unpack (year, branch, sec, roll).
Check year is 2 digits and between 20 and 30.
Check branch is in permitted engineering branch list.
Check section is in permitted section list [A, B, C, D].
Check roll is 4 numeric digits.
If errors list is empty: return (True, extracted_tuple, []).
Else: return (False, None, errors).
Step 4: Function audit_student_credentials(records):
Iterate over student tuples, evaluate email and registration ID, tabulate results.
Step 5: Output formatted verification table and itemized diagnostic audit trail.
Step 6: Stop.

---
```

6. Textbook Pseudocode

```
BEGIN StudentCredentialValidator
INPUT records_list
SET valid_branches = ["CSE", "ECE", "EEE", "MECH", "CIVIL", "IT", "CSM", "CSD"]
SET valid_sections = ["A", "B", "C", "D"]

FOR EACH (email, reg_id) IN records_list DO
SET email_errors = []
SET reg_errors = []

// Email checks
IF COUNT(email, "@") != 1 THEN
APPEND "Email must contain exactly one @" TO email_errors
ELSE
SET parts = SPLIT(email, "@")
SET local_p = parts[0]
SET domain_p = parts[1]

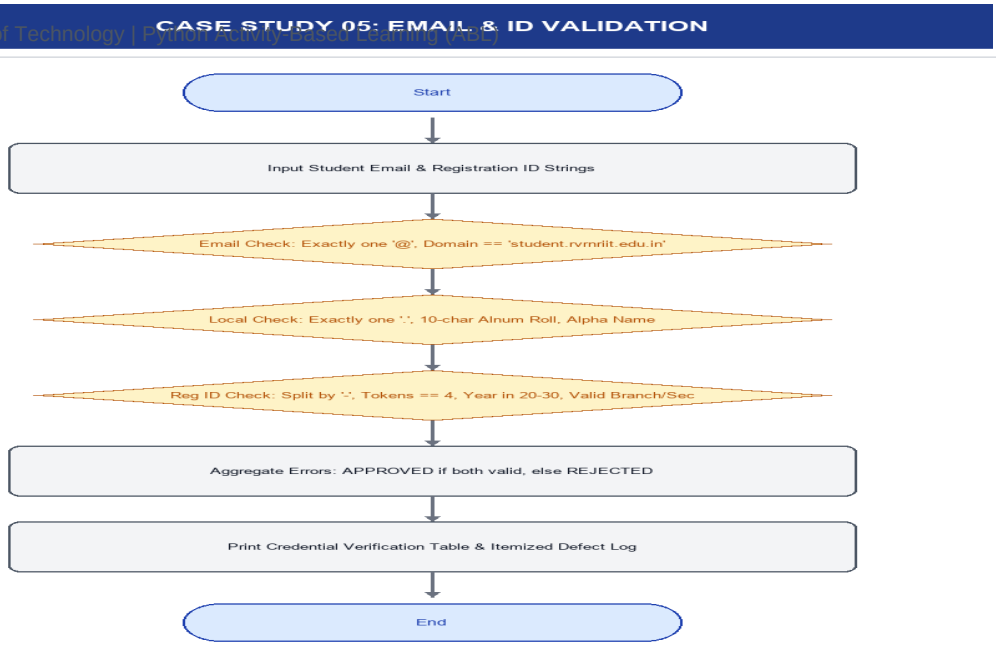
IF domain_p != "student.rvrnriit.edu.in" THEN
APPEND "Domain mismatch" TO email_errors
END IF

IF COUNT(local_p, ".") != 1 THEN
APPEND "Local part must contain single dot separator" TO email_errors
ELSE
SET ltokens = SPLIT(local_p, ".")
IF LENGTH(ltokens[0]) != 10 OR NOT IS_ALNUM(ltokens[0]) THEN
APPEND "Invalid 10-char alphanumeric roll token" TO email_errors
END IF
IF LENGTH(ltokens[1]) < 2 OR NOT IS_ALPHA(ltokens[1]) THEN
APPEND "Invalid student name token" TO email_errors
```

10. Conclusion & Analytical Insights

This case study proves that complex syntax verification and string tokenization can be performed accurately using core string methods, conditional assertions, and character scanning without the regular expressions engine.

5. Architectural Flowchart Diagram



7. Python Source Code Implementation (problem_30_email_identifier_validation.py)

```
"""
Case Study 05: Student Email and Registration Identifier Validation Engine
================================================================================
Syllabus Concepts Used:
- Strings (indexing, slicing, character iteration, traversal)
- String Methods (split, count, startswith, endswith, isalpha, isdigit, isalnum)
- Variables, Data Types (str, bool, list, tuple)
- Relational and Logical Operators
- Conditional Statements (if, elif, else)
- Loops (for, while)
- Lists and Tuples (accumulating error messages, unpacking)
- String Formatting

Description:
Validates institutional email addresses and university student registration identifiers
WITHOUT using regular expressions (re module). Implements rigorous deterministic
string-parsing automata to detect:
- Malformed or misplaced separators (@, ., -)
- Missing prefix or suffix components
- Invalid character categories (symbols where letters/digits required)
- Invalid branch codes and year ranges
"""

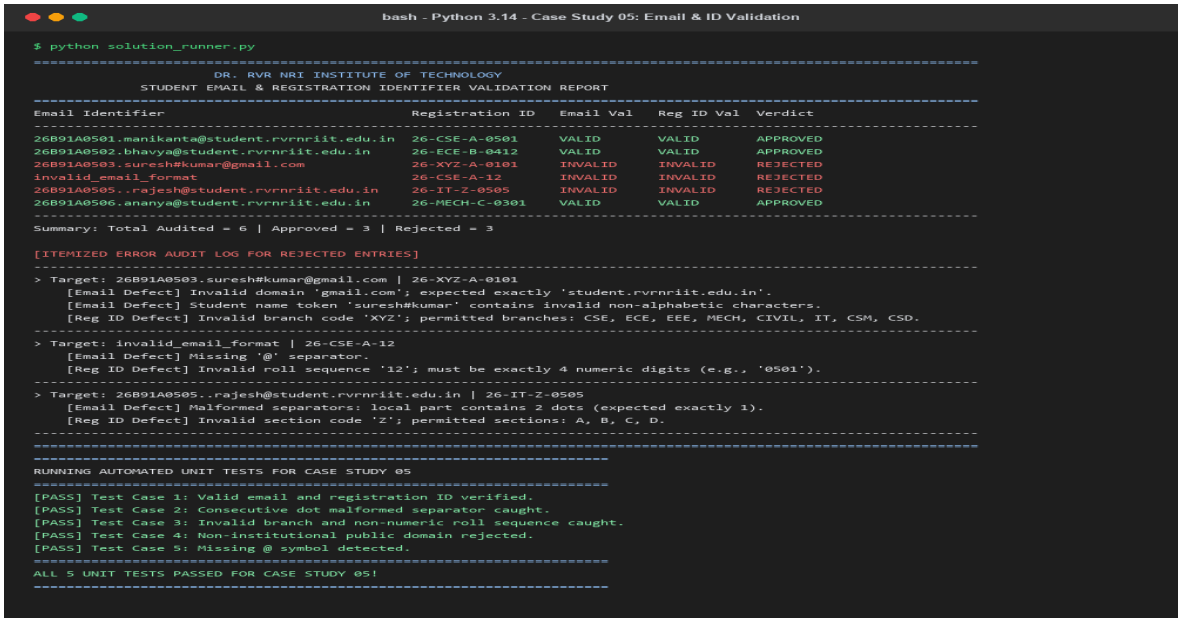
VALID_BRANCHES = ["CSE", "ECE", "EEE", "MECH", "CIVIL", "IT", "CSM", "CSD"]
VALID_SECTIONS = ["A", "B", "C", "D"]
EXPECTED_DOMAIN = "student.rvrnriit.edu.in"

def validate_student_email(email_str):
    """
    Validates institutional student email format:
    <roll_number>.<student_name>@student.rvrnriit.edu.in
    Returns tuple: (is_valid, extracted_info_tuple, errors_list)
    """
    errors = []
    email = email_str.strip()

    # Rule 1: Whitespace check
    for ch in email:
        if ch == " " or ch == "\t" or ch == "\n":
            errors.append("Email contains illegal whitespace characters.")
            break

    # Rule 2: Exactly one '@' symbol
    at_count = email.count("@")
    if at_count == 0:
... [Complete 201 lines available in problem_30_email_identifier_validation.py]
```

8 & 9. Test Verification & Terminal Execution Output



CASE STUDY 06: STUDENT MARKS MATRIX ANALYSIS SYSTEM

Faculty Prescribed Problem #42 | Concepts: 2D Nested Lists, Row-Major / Column-Major Traversal, Extremums

1. Problem Statement

Write a Python program to analyse marks stored in a two-dimensional list. It finds totals, percentages, grades, subject averages, and the students with the highest marks. ---

2. Objectives

• Store the marks in a nested list.

• Find each student's total, average, grade, and pass or fail result.

• Find the average and pass percentage for every subject.

• Show subjects that may need more practice.

• --

3. Assumptions

1. Assessment marks are organized in a 2D matrix (nested list): `marks_matrix[student_index][subject_index]`.

2. Each subject has a maximum achievable score of 100 marks.

3. Minimum passing score in any individual subject is 40 marks.

4. Grading criteria (based on percentage, provided all subjects are passed):

- $\ge 85\%$: O (Outstanding)

- $75\% \text{ to } 84.9\%$: A+ (Excellent)

- $65\% \text{ to } 74.9\%$: A (Very Good)

- $55\% \text{ to } 64.9\%$: B (Good)

- $40\% \text{ to } 54.9\%$: C (Pass)

- $< 40\%$ or any subject failed: F (Fail)

5. A subject is flagged for institutional review if its average score $< 65.0\%$ or pass percentage $< 70.0\%$.

---

4. Step-by-Step Algorithm

```
Step 1: Start.
Step 2: Function analyze_marks_matrix(students, subjects, marks_matrix):
Set num_students = len(students), num_subjects = len(subjects).
Initialize student_results = [], highest_total = -1, top_performer = None.
// Row-wise traversal
For i from 0 to num_students - 1:
Set row_marks = marks_matrix[i], row_total = 0, all_passed = True.
For each mark m in row_marks:
row_total += m
If m < 40: all_passed = False
avg = row_total / num_subjects.
grade = determine_grade(avg, all_passed).
status = "PASSED" if all_passed else "FAILED".
Append (roll, name, row_marks, row_total, avg, grade, status) to student_results.
If all_passed and row_total > highest_total:
highest_total = row_total, top_performer = (roll, name, row_total, avg).
// Column-wise traversal
Initialize subject_stats = [], low_performing_subjects = [].
For j from 0 to num_subjects - 1:
Set col_sum = 0, col_max = -1, col_min = 1000, pass_count = 0.
For i from 0 to num_students - 1:
score = marks_matrix[i][j]
col_sum += score
If score > col_max: col_max = score
If score < col_min: col_min = score
If score >= 40: pass_count += 1
sub_avg = col_sum / num_students.
sub_pass_pct = (pass_count / num_students) * 100.0.
stat_tuple = (subjects[j], sub_avg, col_max, col_min, sub_pass_pct).
Append stat_tuple to subject_stats.
If sub_avg < 65.0 or sub_pass_pct < 70.0:
Append stat_tuple to low_performing_subjects.
Return student_results, subject_stats, top_performer, low_performing_subjects.
Step 3: Print formatted 2D assessment grid, subject statistics, and audit conclusions.
Step 4: Stop.

---
```

6. Textbook Pseudocode

```
BEGIN MarksMatrixAnalysis
INPUT students, subjects, matrix
SET M = LENGTH(students)
SET N = LENGTH(subjects)

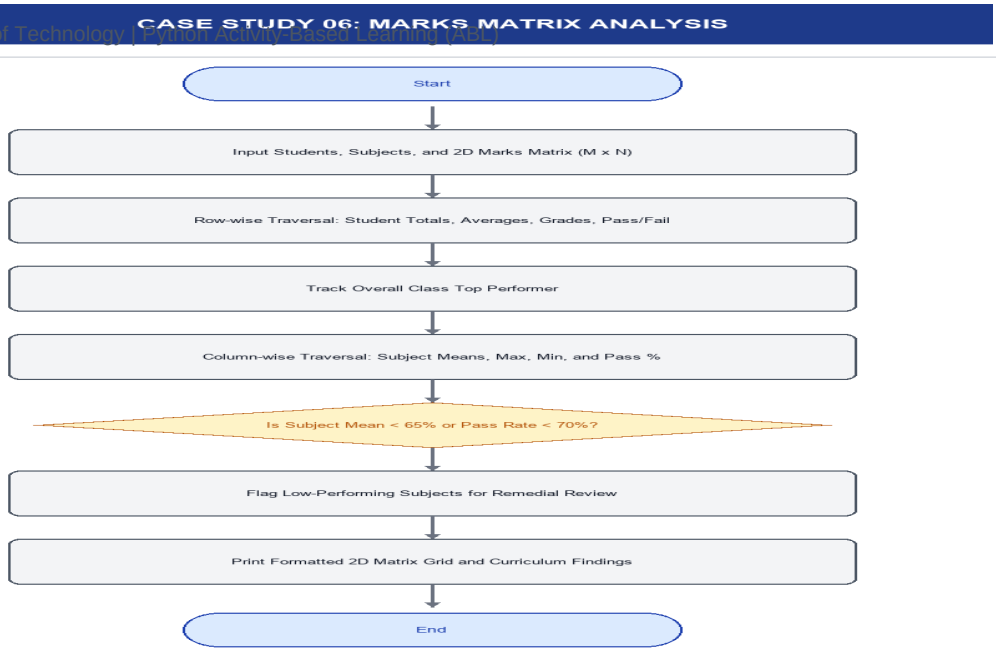
// Row-Wise Processing
FOR i FROM 0 TO M - 1 DO
SET row_sum = 0
SET passed = TRUE
FOR j FROM 0 TO N - 1 DO
SET score = matrix[i][j]
row_sum = row_sum + score
IF score < 40 THEN
passed = FALSE
END IF
END FOR
SET avg = row_sum / N
SET grade = GET_GRADE(avg, passed)
SET status = passed ? "PASSED" : "FAILED"
STORE (students[i], row_sum, avg, grade, status)
END FOR

// Column-Wise Processing
FOR j FROM 0 TO N - 1 DO
SET col_sum = 0
SET mx = -1
SET mn = 1000
SET pass_cnt = 0
FOR i FROM 0 TO M - 1 DO
SET val = matrix[i][j]
```

10. Conclusion & Analytical Insights

Nested lists provide a natural representation for multidimensional tabular structures. Navigating 2D data via row-major and column-major loops delivers comprehensive academic insights without dependence on external data science packages.

5. Architectural Flowchart Diagram



7. Python Source Code Implementation (problem_42_marks_matrix_analysis.py)

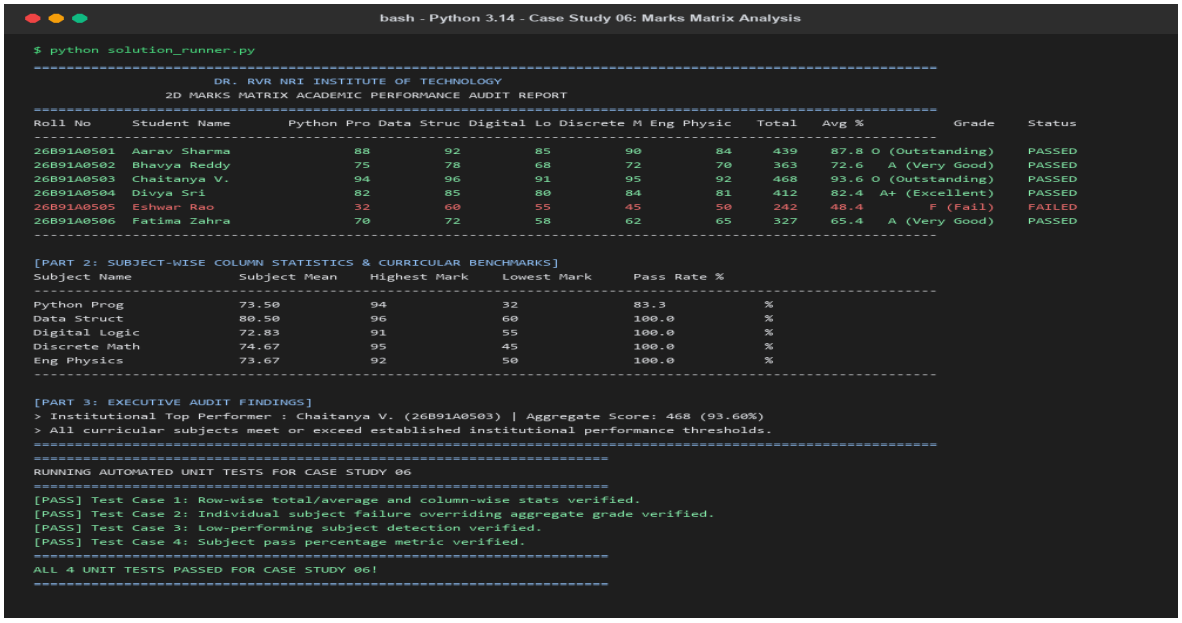
```
"""
Case Study 06: Student Marks Matrix Analysis System
==================================================
Syllabus Concepts Used:
- Variables, Data Types (int, float, str, bool)
- Relational and Logical Operators
- Conditional Statements (if, elif, else)
- Nested Loops (row-wise and column-wise matrix traversals)
- Nested Lists (2D Grid Representation)
- Tuples and Tuple Unpacking
- String Formatting for 2D Tabular Display

Description:
Performs 2-dimensional matrix computation on academic assessment marks:
1. Row-Wise Analytics: Student totals, averages, percentages, letter grades,
   and pass/fail determinations (pass mark = 40 per subject).
2. Column-Wise Analytics: Subject means, highest marks, lowest marks,
   and subject-specific pass percentages.
3. Class performance summary: Finds the students with the highest marks
   and flags curricular subjects with below-benchmark averages (< 65.0%).
"""

def determine_grade(pct, passed_all):
    if not passed_all:
        return "F (Fail)"
    if pct >= 85.0:
        return "O (Outstanding)"
    elif pct >= 75.0:
        return "A+ (Excellent)"
    elif pct >= 65.0:
        return "A (Very Good)"
    elif pct >= 55.0:
        return "B (Good)"
    elif pct >= 40.0:
        return "C (Pass)"
    else:
        return "F (Fail)"


def analyze_marks_matrix(students, subjects, marks_matrix):
    """
    students: List of tuples (roll_no, name)
    subjects: List of strings [sub1, sub2, sub3, sub4, sub5]
    marks_matrix: 2D Nested List of integers [num_students x num_subjects]
    """
... [Complete 182 lines available in problem_42_marks_matrix_analysis.py]
```

8 & 9. Test Verification & Terminal Execution Output



CASE STUDY 07: MULTI-WAREHOUSE INVENTORY MATRIX RECONCILIATION

Faculty Prescribed Problem #46 | Concepts: Nested Lists, Variance Matrix, Greedy Rebalancing, Transfer Tuples

1. Problem Statement

Write a Python program to compare the stock available in different warehouses with the required stock. It shows shortages and extra items, and suggests transfers between warehouses using nested lists. ---

2. Objectives

• Store the actual and required stock in nested lists.

• Find extra stock and shortage for every warehouse.

• Suggest transfers from a warehouse with extra stock to one with a shortage.

• Show products that still need to be purchased.

• --

3. Assumptions

1. Facilities and inventory categories are structured as $M \times N$ matrices:

- M Warehouses: `['WH-1 Central Eluru', 'WH-2 North Vijayawada', 'WH-3 East Guntur', 'WH-4 South Mangalagiri']`.

- N Products: `['Laptops', 'Servers', 'Routers', 'Projectors', 'Smartboards']`.

2. Actual stock and target stock values are non-negative integers.

3. Intra-network transfers carry zero transfer loss; transferred quantities immediately balance target deficits.

4. If total surplus across all warehouses for a product is less than total deficit, remaining unmet quantities require external replenishment.

---

4. Step-by-Step Algorithm

```
Step 1: Start.
Step 2: Function reconcile_inventory(warehouses, products, actual, target):
Set num_wh = len(warehouses), num_prod = len(products).
Initialize diff_matrix = []
For i from 0 to num_wh - 1:
row = []
For j from 0 to num_prod - 1:
row.append(actual[i][j] - target[i][j])
diff_matrix.append(row)
Initialize transfer_schedule = [], unmet_deficits = [].
For j from 0 to num_prod - 1:
prod_name = products[j]
surplus_list = [], deficit_list = [].
For i from 0 to num_wh - 1:
d = diff_matrix[i][j]
If d > 0: surplus_list.append([i, d])
Else if d < 0: deficit_list.append([i, abs(d)])
s_idx = 0, d_idx = 0
While s_idx < len(surplus_list) and d_idx < len(deficit_list):
src_wh, s_qty = surplus_list[s_idx]
dst_wh, d_qty = deficit_list[d_idx]
transfer_qty = min(s_qty, d_qty)
Append (warehouses[src_wh], warehouses[dst_wh], prod_name, transfer_qty) to transfer_schedule.
surplus_list[s_idx][1] -= transfer_qty
deficit_list[d_idx][1] -= transfer_qty
If surplus_list[s_idx][1] == 0: s_idx += 1
If deficit_list[d_idx][1] == 0: d_idx += 1
While d_idx < len(deficit_list):
dst_wh, rem = deficit_list[d_idx]
If rem > 0:
Append (warehouses[dst_wh], prod_name, rem) to unmet_deficits.
d_idx += 1
Return diff_matrix, transfer_schedule, unmet_deficits.
Step 3: Print formatted actual matrix, target matrix, variance matrix, transfer orders, and procurement
alerts.
Step 4: Stop.

---
```

6. Textbook Pseudocode

```
BEGIN InventoryReconciliation
INPUT warehouses, products, actual, target
SET M = LENGTH(warehouses)
SET N = LENGTH(products)

// Compute Variance
FOR i FROM 0 TO M - 1 DO
FOR j FROM 0 TO N - 1 DO
SET diff[i][j] = actual[i][j] - target[i][j]
END FOR
END FOR

SET transfers = []
SET unmet = []

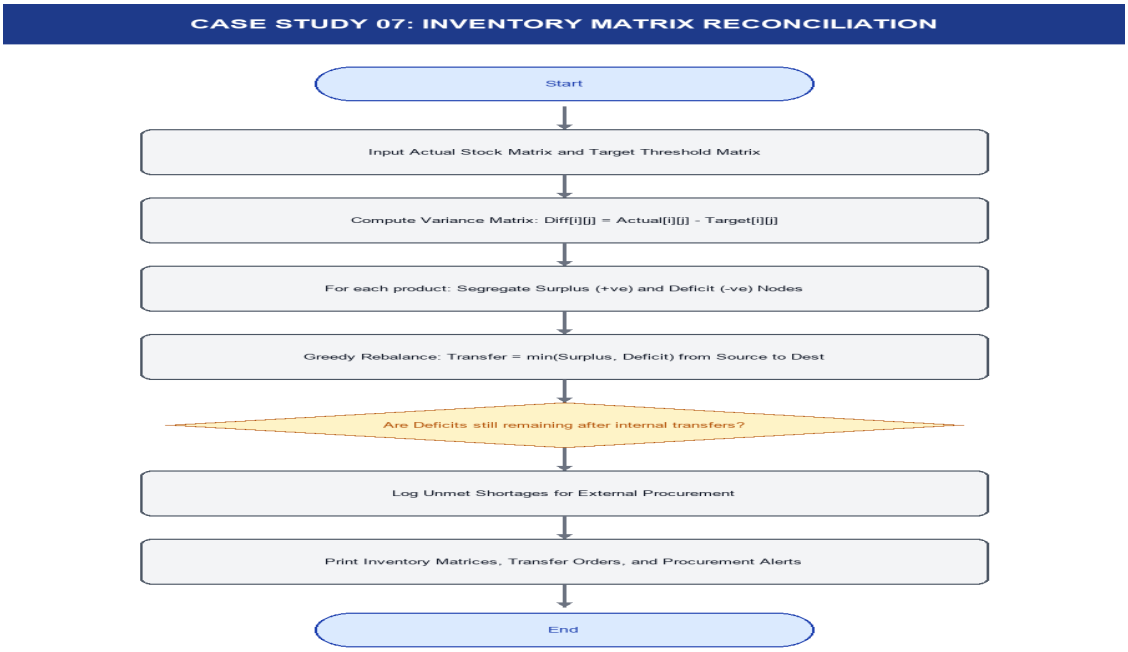
// Greedy Column-Wise Rebalancing
FOR j FROM 0 TO N - 1 DO
SET surpluses = []
SET deficits = []
FOR i FROM 0 TO M - 1 DO
IF diff[i][j] > 0 THEN APPEND [i, diff[i][j]] TO surpluses END IF
IF diff[i][j] < 0 THEN APPEND [i, -diff[i][j]] TO deficits END IF
END FOR

SET s = 0, d = 0
WHILE s < LENGTH(surpluses) AND d < LENGTH(deficits) DO
SET transfer_qty = MIN(surpluses[s][1], deficits[d][1])
APPEND (warehouses[surpluses[s][0]], warehouses[deficits[d][0]], products[j], transfer_qty) TO transfers
surpluses[s][1] = surpluses[s][1] - transfer_qty
deficits[d][1] = deficits[d][1] - transfer_qty
```

10. Conclusion & Analytical Insights

This case study demonstrates multi-dimensional nested list manipulation applied to real-world supply-chain optimization. Pointer-based greedy inventory transfer logic seamlessly resolves intra-network imbalances while identifying external procurement requirements.

5. Architectural Flowchart Diagram



7. Python Source Code Implementation (problem_46_inventory_reconciliation.py)

```
"""
Case Study 07: Multi-Warehouse Inventory Matrix Reconciliation System
======================================================================
Syllabus Concepts Used:
- Nested Lists (2D Warehouse x Product Matrix)
- Variables, Data Types (int, str, list, tuple)
- Relational and Logical Operators
- Conditional Statements (if, elif, else)
- Nested Loops (row-wise and column-wise traversals)
- Tuples and Tuple Unpacking
- Greedy Rebalancing Algorithm without External Libraries
- Formatted Tabular Statements

Description:
Reconciles product stock across multiple regional warehouses:
1. Computes the deviation matrix (Actual Stock - Target Stock).
2. Classifies warehouse-product pairs into Surpluses, Deficits, or Balanced.
3. Computes automated inter-warehouse stock transfer dispatches:
   (source_wh, dest_wh, product_name, transfer_qty)
4. Determines post-transfer network balance and unmet supply deficits.
"""

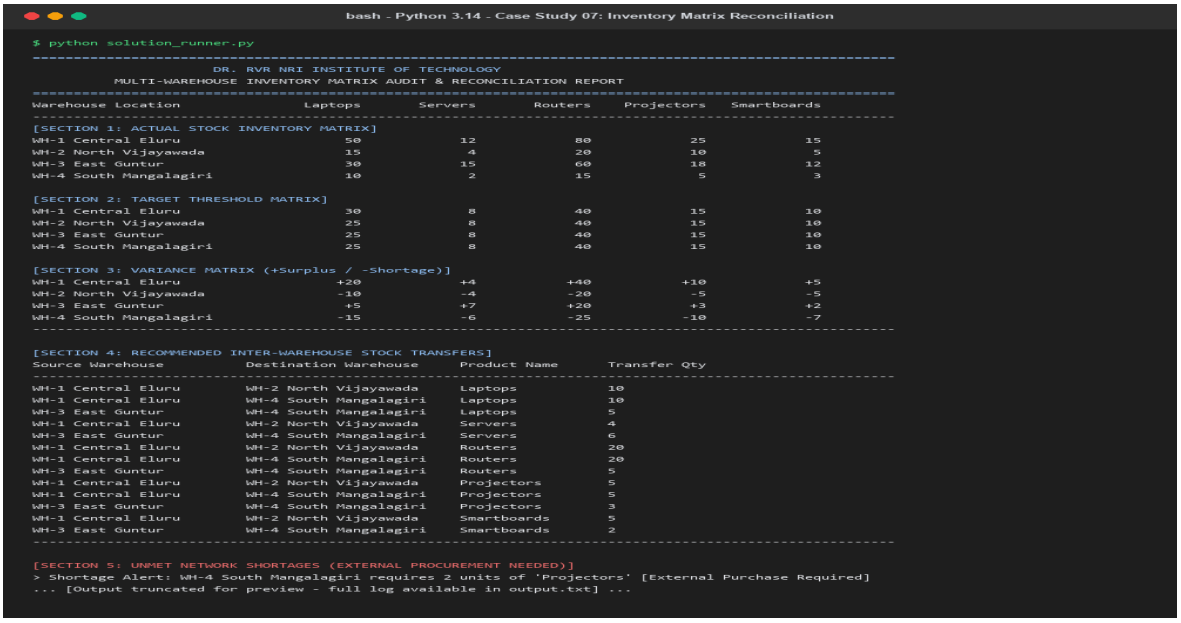
def reconcile_inventory(warehouses, products, actual_matrix, target_matrix):
    """
    warehouses: List of strings (warehouse names)
    products: List of strings (product names)
    actual_matrix: 2D nested list [num_wh x num_products]
    target_matrix: 2D nested list [num_wh x num_products]
    """
    num_wh = len(warehouses)
    num_prod = len(products)

    # 1. Compute difference matrix (Actual - Target)
    diff_matrix = []
    for i in range(num_wh):
        row_diff = []
        for j in range(num_prod):
            row_diff.append(actual_matrix[i][j] - target_matrix[i][j])
        diff_matrix.append(row_diff)

    # 2. Product-wise transfer planning
    transfer_schedule = []
    unmet_deficits = []

    for j in range(num_prod):
... [Complete 170 lines available in problem_46_inventory_reconciliation.py]
```

8 & 9. Test Verification & Terminal Execution Output



CASE STUDY 08: TUPLE-BASED EMPLOYEE SHIFT CONFLICT DETECTION

Faculty Prescribed Problem #48 | Concepts: Shift Tuples, Unpacking, 24-hr Time Conversion, Interval Intersections

1. Problem Statement

Write a Python program to store employee shifts as tuples and find overlapping shifts. It also checks whether an employee has enough rest time between two shifts. ---

2. Objectives

• Store every shift as a tuple.

• Convert a time such as `08:30` into minutes.

• Compare two shifts for the same employee and day.

• Report overlapping shifts and breaks shorter than eight hours.

• --

3. Assumptions

1. Shifts are immutable tuples adhering strictly to 24-hour format: `"HH:MM"` (e.g., `"08:00"` to `"16:30"`).

2. Shifts are defined within single calendar days (00:00 to 23:59).

3. A conflict occurs when:

- Same `emp_id` and same `day_of_week`.

- Direct overlap: $\max(\text{start}_1, \text{start}_2) < \min(\text{end}_1, \text{end}_2)$.

- Insufficient recovery: $0 \le (\text{start}_{\text{later}} - \text{end}_{\text{earlier}}) < 480 \text{ minutes}$.

4. Roster processing handles multiple campus departments (Labs, Server Room, Workshops, Library).

---

4. Step-by-Step Algorithm

```
Step 1: Start.
Step 2: Function parse_time_to_minutes(time_str):
Split string by ':' into hours and minutes.
Return hours * 60 + minutes.
Step 3: Function detect_shift_conflicts(shifts, min_rest_hours=8):
Set min_rest_minutes = min_rest_hours * 60.
Initialize direct_overlaps = [], rest_violations = [].
Set n = len(shifts).
For i from 0 to n - 1:
For j from i + 1 to n - 1:
Unpack shifts[i] into (s1_id, e1_id, name1, day1, st1_str, et1_str, dept1).
Unpack shifts[j] into (s2_id, e2_id, name2, day2, st2_str, et2_str, dept2).
If e1_id == e2_id and day1.lower() == day2.lower():
Convert start and end strings to minutes: st1, et1, st2, et2.
Compute overlap_start = max(st1, st2).
Compute overlap_end = min(et1, et2).
If overlap_start < overlap_end:
duration = overlap_end - overlap_start
Append collision record to direct_overlaps.
Else:
If et1 <= st2: gap = st2 - et1, earlier = s1_id, later = s2_id
Else: gap = st1 - et2, earlier = s2_id, later = s1_id
If 0 <= gap < min_rest_minutes:
Append violation record to rest_violations.
Return direct_overlaps, rest_violations.
Step 4: Output formatted roster overview, direct overlap alerts, and insufficient rest warnings.
Step 5: Stop.

---
```

6. Textbook Pseudocode

```
BEGIN ShiftConflictDetection
INPUT shifts_list, min_rest_hours
SET min_rest = min_rest_hours * 60
SET overlaps = []
SET rest_alerts = []
SET N = LENGTH(shifts_list)

FOR i FROM 0 TO N - 1 DO
FOR j FROM i + 1 TO N - 1 DO
UNPACK shifts_list[i] INTO (id1, e1, name1, day1, st1_s, et1_s, d1)
UNPACK shifts_list[j] INTO (id2, e2, name2, day2, st2_s, et2_s, d2)

IF e1 == e2 AND LOWER(day1) == LOWER(day2) THEN
SET st1 = PARSE_MINUTES(st1_s)
SET et1 = PARSE_MINUTES(et1_s)
SET st2 = PARSE_MINUTES(st2_s)
SET et2 = PARSE_MINUTES(et2_s)

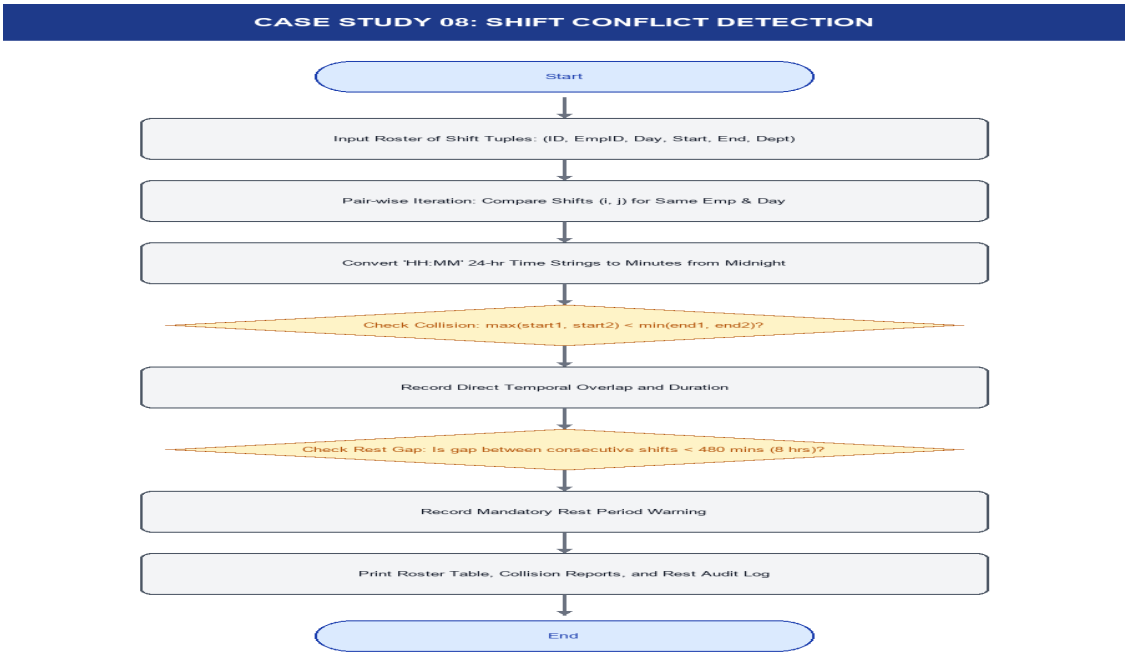
SET ov_start = MAX(st1, st2)
SET ov_end = MIN(et1, et2)

IF ov_start < ov_end THEN
SET dur = ov_end - ov_start
APPEND (e1, name1, day1, id1, id2, dur) TO overlaps
ELSE
IF et1 <= st2 THEN
SET gap = st2 - et1
SET p_id = id1, n_id = id2
ELSE
SET gap = st1 - et2
```

10. Conclusion & Analytical Insights

This case study highlights the efficacy of tuple data structures and unpacking in enforcing operational integrity. Mathematical time conversions and interval overlap checks prevent scheduling collisions and protect institutional employee welfare.

5. Architectural Flowchart Diagram



7. Python Source Code Implementation (problem_48_shift_conflict_detection.py)

```
"""
Case Study 08: Tuple-Based Employee Shift Conflict Detection Engine
======================================================================
Syllabus Concepts Used:
- Tuples and Tuple Packing/Unpacking
- Tuple Operations (indexing, concatenation, comparison)
- String Slicing and String Methods (split, strip)
- Variables, Data Types (str, int, bool, list, tuple)
- Relational and Logical Operators
- Conditional Statements (if, elif, else)
- Nested Loops for Pair-Wise Shift Comparisons
- Formatted Tabular Reports

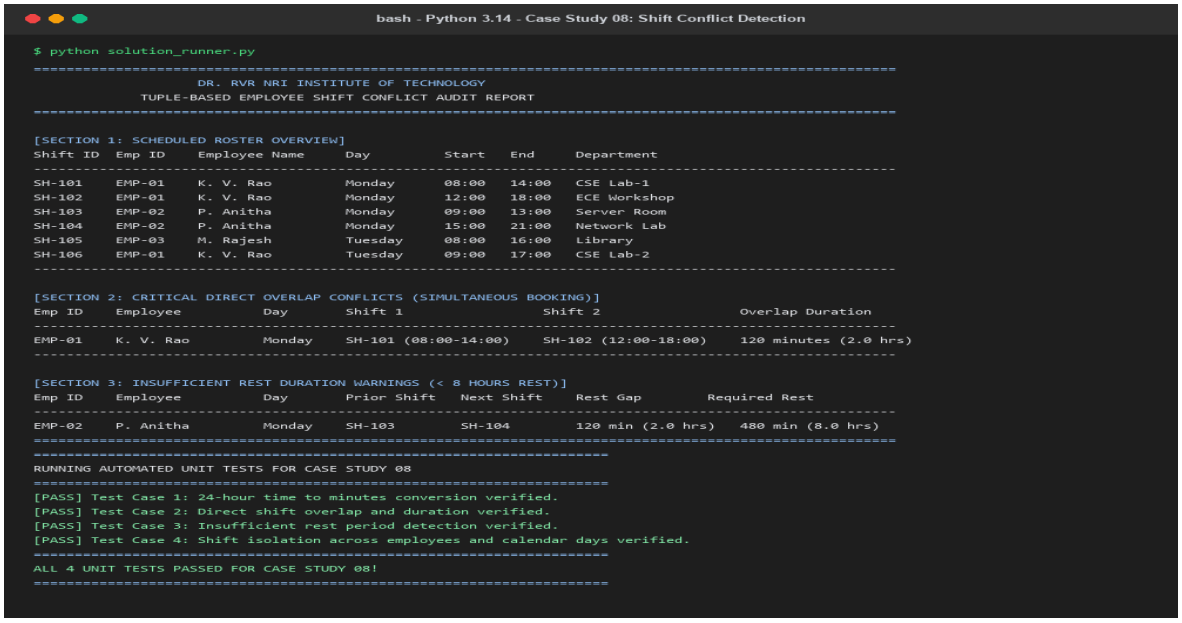
Description:
Audits institutional duty rotas to detect shift schedule conflicts:
1. Converts 24-hour time strings ("HH:MM") to total minutes from midnight.
2. Identifies direct temporal overlaps (where an employee is scheduled simultaneously).
3. Identifies insufficient rest intervals (< 8 hours / 480 minutes between duties).
4. Identifies cross-departmental double bookings.
"""

def parse_time_to_minutes(time_str):
    """
    Converts 'HH:MM' 24-hour string format into minutes since midnight.
    e.g., '08:30' -> 8 * 60 + 30 = 510 minutes.
    """
    clean_str = time_str.strip()
    parts = clean_str.split(":")
    hours = int(parts[0])
    minutes = int(parts[1])
    return hours * 60 + minutes


def format_minutes_to_time(minutes):
    """
    Converts minutes back to 'HH:MM' string format.
    """
    hrs = minutes // 60
    mins = minutes % 60
    return f"{hrs:02d}:{mins:02d}"


def detect_shift_conflicts(shifts, min_rest_hours=8):
    """
    shifts: List of tuples
... [Complete 154 lines available in problem_48_shift_conflict_detection.py]
```

8 & 9. Test Verification & Terminal Execution Output



CASE STUDY 09: CAMPUS FESTIVAL REGISTRATION & FINANCIAL AUDIT

Faculty Prescribed Problem #74 | Concepts: Duplicate Key Detection, Lists & Tuples, Budget Variance, Balance Sheet

1. Problem Statement

Write a Python program to record festival registrations, sponsorships, and expenses. The program finds duplicate registrations, compares the budget with the actual spending, and prints the final balance. ---

2. Objectives

• **Data Integration Across Disparate Streams:** Aggregate registration tuples, sponsorship tuples, and multi-category expense nested lists.

• **Duplicate Registration Filtration:** Detect duplicate event entries ($(\text{Student ID}, \text{Event Code})$ tuples) using list-based uniqueness tracking without external hashing libraries or sets.

• **Sponsorship Realization Analysis:** Reconcile committed contract values against realized payments across corporate sponsorship tiers.

• **Budgetary Variance Auditing:** Calculate expense variances $(\text{Budget} - \text{Actual})$ per operational category and flag cost overruns.

• **Institutional Net Worth Statement:** Deliver a final balance sheet reflecting Net Surplus or Deficit.

• --

3. Assumptions

1. Registrations are tuples:

`(reg_id, student_id, student_name, event_code, fee_paid, payment_status)`

2. A duplicate occurs when the same `student_id` registers for the same `event_code` multiple times.

3. Corporate sponsorships are tuples:

`(sponsor_id, company_name, tier, committed_amount, received_amount)`

4. Operational expenses are stored in a nested list:

`[[category, item_desc, budgeted_amount, actual_amount], ...]`

5. Total Inflow = Verified Paid Registrations + Realized Sponsorships.

6. Net Position = Total Inflow - Actual Total Expenditures.

---

4. Step-by-Step Algorithm

```
Step 1: Start.
Step 2: Function audit_festival_finances(registrations, sponsorships, expenses):
Initialize seen_pairs = [], valid_regs = [], dup_regs = [].
Initialize total_reg_collected = 0.0, pending_fees = 0.0.
// Registration Audit
For reg in registrations:
Unpack (reg_id, s_id, s_name, ev_code, fee, status).
key = s_id + "::" + ev_code.
If key in seen_pairs:
Append reg to dup_regs.
Else:
Append key to seen_pairs.
If status == "PAID":
Append reg to valid_regs.
total_reg_collected += fee.
Else:
pending_fees += fee.
// Sponsorship Audit
Initialize total_comm = 0, total_rec = 0, shortfalls = [].
For sp in sponsorships:
Unpack (sp_id, comp, tier, committed, received).
total_comm += committed, total_rec += received.
If committed > received:
Append shortfall record to shortfalls.
// Expense Audit
Initialize total_budget = 0, total_actual = 0, exp_summary = [].
For exp in expenses:
Unpack [cat, item, budgeted, actual].
total_budget += budgeted, total_actual += actual.
variance = budgeted - actual.
is_over = (actual > budgeted).
Append (cat, item, budgeted, actual, variance, is_over) to exp_summary.
// Net Position
total_income = total_reg_collected + total_rec.
net_balance = total_income - total_actual.
is_surplus = (net_balance >= 0).
Return audit_data.
Step 3: Print structured audit report tables: Registrations & Duplicates, Sponsorships, Budget Variances,
and Balance Sheet.
Step 4: Stop.

---
```

6. Textbook Pseudocode

```
BEGIN CampusFestivalFinancialAudit
INPUT registrations, sponsorships, expenses
SET seen_keys = []
SET reg_rev = 0, pending_rev = 0
SET dup_list = []

FOR EACH reg IN registrations DO
UNPACK reg INTO (rid, sid, sname, ev, fee, stat)
SET key = sid + ":" + ev
IF key IN seen_keys THEN
APPEND reg TO dup_list
ELSE
APPEND key TO seen_keys
IF stat == "PAID" THEN
reg_rev = reg_rev + fee
ELSE
pending_rev = pending_rev + fee
END IF
END IF
END FOR

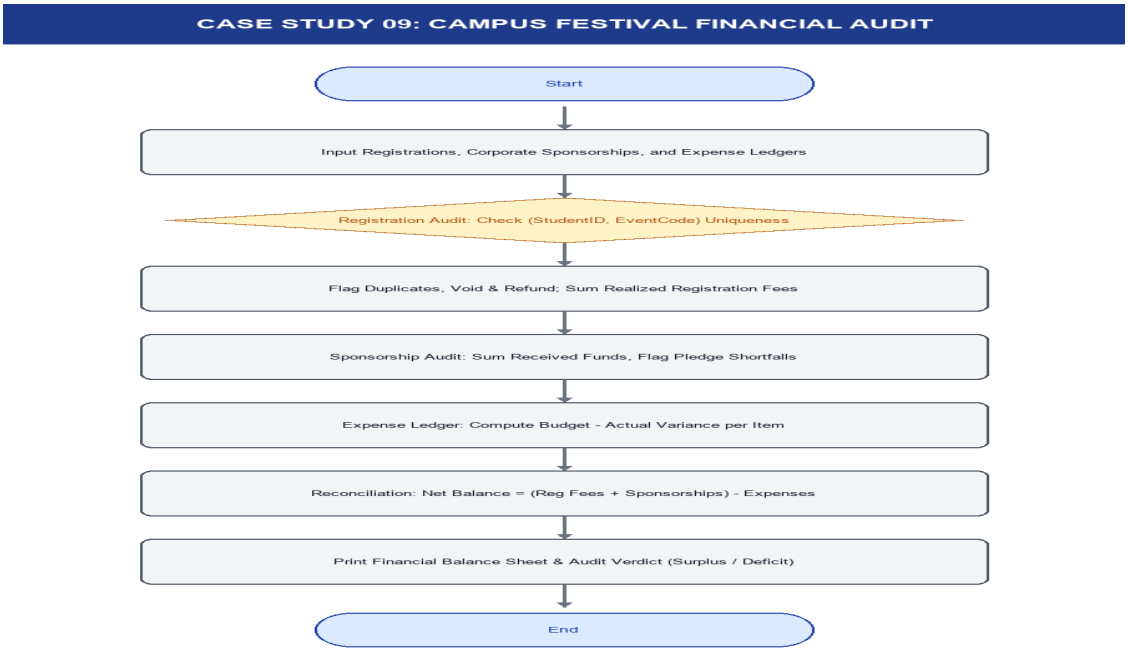
SET spon_rev = 0
FOR EACH sp IN sponsorships DO
UNPACK sp INTO (spid, comp, tier, comm, rec)
spon_rev = spon_rev + rec
END FOR

SET tot_exp = 0
FOR EACH exp IN expenses DO
UNPACK exp INTO (cat, item, bud, act)
```

10. Conclusion & Analytical Insights

This case study illustrates how multi-source institutional financial ledgers are consolidated and audited using lists, tuples, and nested arrays. Computational tracking prevents financial leakages, identifies vendor overruns, and ensures full financial accountability.

5. Architectural Flowchart Diagram



7. Python Source Code Implementation (problem_74_festival_audit.py)

```
"""
Case Study 09: Campus Festival Registration and Financial Audit System
=======================================================================
Syllabus Concepts Used:
- Lists and Nested Lists (Multi-Category Expense Ledgers)
- Tuples, Tuple Unpacking, and Tuple Concatenation
- String Slicing and Formatting
- Variables and Numerical Types (float, int, str, bool)
- Relational and Logical Operators
- Conditional Statements (if, elif, else)
- Nested Loops and Membership Iteration for Duplicate Detection
- Formatted Financial Statements and Auditing Reports

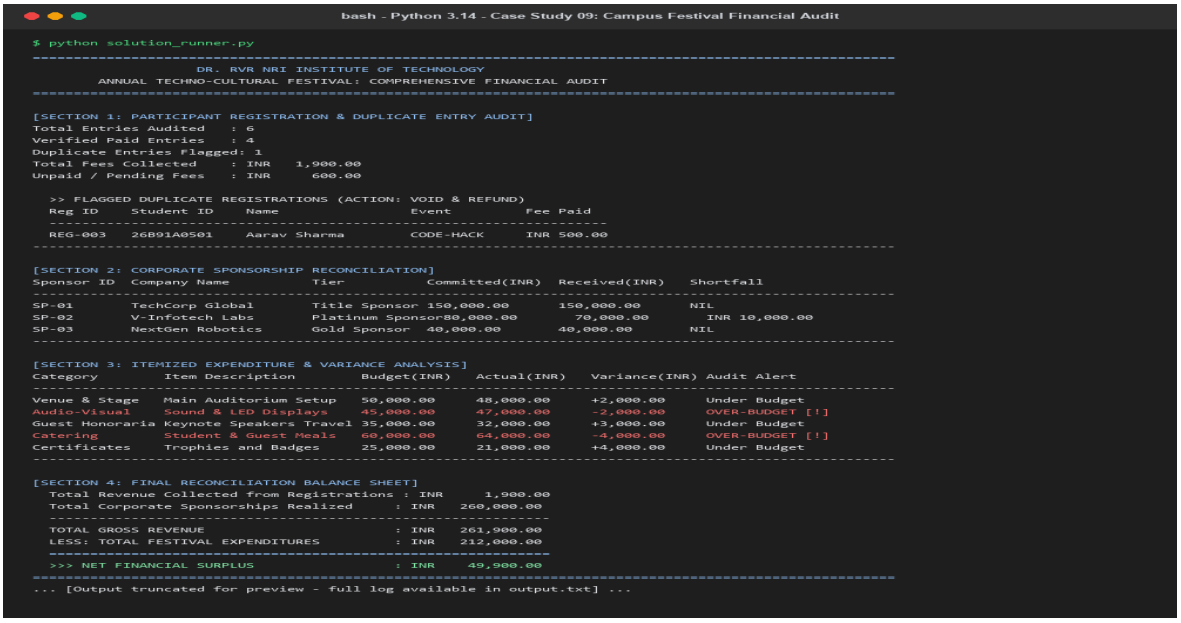
Description:
Reconciles festival accounts for the annual university techno-cultural fest:
1. Audits participant registrations: flags duplicates (same student + same event)
   and isolates verified collected registration revenue.
2. Audits corporate sponsorship contributions across tiers (committed vs received).
3. Reconciles itemized expenditures against allocated budgets.
4. Computes net fiscal surplus / deficit and compiles an audit certificate.
"""

def audit_festival_finances(registrations, sponsorships, expenses):
    """
    registrations: List of tuples
      (reg_id, student_id, student_name, event_code, fee_paid, status)
    sponsorships: List of tuples
      (sponsor_id, company_name, tier, committed_amount, received_amount)
    expenses: Nested list
      [[category, item_desc, budgeted_amount, actual_amount], ...]
    """
    # 1. Audit Registrations and Detect Duplicates
    seen_student_events = []
    valid_registrations = []
    duplicate_registrations = []
    total_reg_collected = 0.0
    pending_reg_fees = 0.0

    for reg in registrations:
        reg_id, s_id, s_name, ev_code, fee, status = reg
        pair_key = s_id.upper() + "::" + ev_code.upper()

        if pair_key in seen_student_events:
            duplicate_registrations.append(reg)
        else:
... [Complete 188 lines available in problem_74_festival_audit.py]
```

8 & 9. Test Verification & Terminal Execution Output



CASE STUDY 10: SIMPLE SMART CAMPUS MENU

Faculty Prescribed Problem #100 | Concepts: Menu Driven (While Loop), 2D Lab Grid, Summary

1. Problem Statement

Write a menu-driven Python program for a small campus. It includes student details, attendance, marks, event registration, and a computer lab seating grid. ---

2. Objectives

• Keep student, attendance, marks, event, and lab data together.

• Use a menu and a `while` loop to select an operation.

• Check attendance, marks, duplicate registrations, and lab seats.

• Print a small summary of the campus data.

• --

3. Assumptions

1. Student roster is stored as a list of tuples: `(roll_no, name, branch, section)`.

2. Attendance is tracked per student: `(roll_no, total_classes, attended_classes)`. Shortage threshold is $< 75.0\%$.

3. Marks matrix is a 2D nested list with passing benchmark ≥ 40 in all subjects.

4. Events are tuples: `(event_code, event_name, capacity, enrolled_list)`.

5. Lab seats are modeled as a 3×4 nested grid of strings (`"."` for empty, Roll No for occupied).

6. The application supports both interactive terminal usage and headless automated validation.

---

4. Step-by-Step Algorithm

```
Step 1: Start.
Step 2: Initialize campus datasets (students, attendance, subjects, marks, events, lab_seats).
Step 3: Define menu loop:
Display 7 options:
1. Student Registry -> display formatted table.
2. Attendance Monitoring -> compute attendance %; if < 75%, flag shortage.
3. Marks Evaluation -> traverse 2D marks matrix; compute student totals and pass/fail.
4. Event Rosters & Registration -> display capacities; if registering, verify event exists, student is
not already enrolled, and capacity is available.
5. Lab Seating Allocation -> print 2D grid; allocate seat at (row, col) if coordinates are valid, seat is
empty, and student is not seated elsewhere.
6. Executive Dashboard -> aggregate total enrolled, attendance shortages, academic failures, events
count, and lab seat vacancy statistics.
7. Exit -> terminate loop.
Step 4: Stop.

---
```

6. Textbook Pseudocode

```
BEGIN SmartCampusManagementSimulation
INITIALIZE students, attendance, subjects, marks, events, lab_seats

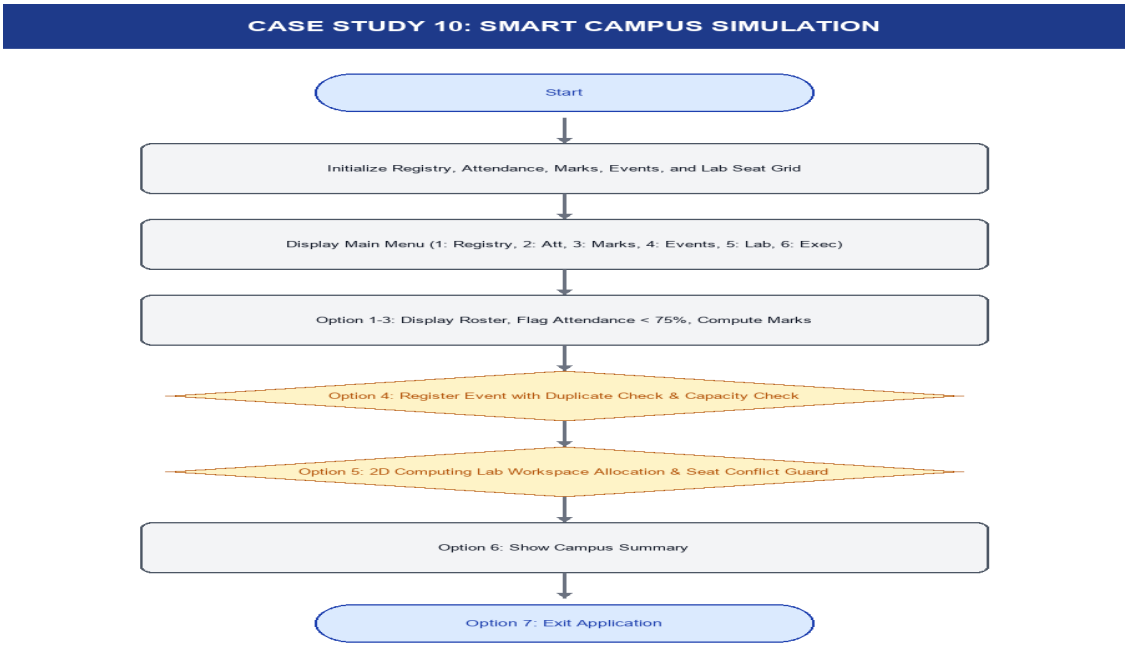
WHILE TRUE DO
DISPLAY_MENU()
INPUT choice

IF choice == 1 THEN
DISPLAY_STUDENT_REGISTRY(students)
ELSE IF choice == 2 THEN
FOR EACH (roll, tot, att) IN attendance DO
SET pct = (att / tot) * 100
IF pct < 75.0 THEN FLAG_SHORTAGE(roll, pct) END IF
END FOR
ELSE IF choice == 3 THEN
FOR i FROM 0 TO LENGTH(students) - 1 DO
SET total = SUM(marks[i])
SET avg = total / LENGTH(subjects)
SET status = ALL(m >= 40 FOR m IN marks[i]) ? "PASS" : "FAIL"
DISPLAY_MARKS_ROW(students[i], marks[i], total, avg, status)
END FOR
ELSE IF choice == 4 THEN
INPUT ev_code, roll_no
REGISTER_EVENT(events, ev_code, roll_no)
ELSE IF choice == 5 THEN
DISPLAY_GRID(lab_seats)
INPUT row, col, roll_no
ALLOCATE_SEAT(lab_seats, row, col, roll_no)
ELSE IF choice == 6 THEN
RENDER_DASHBOARD(students, attendance, marks, events, lab_seats)
```

10. Conclusion & Analytical Insights

This capstone simulation synthesizes all foundational constructs of the assigned Python programming syllabus into a cohesive university management application. Demonstrating robust state management, 2D grid allocation, error shielding, and multi-tier auditing without object-oriented programming or external frameworks highlights the power of structured programming.

5. Architectural Flowchart Diagram



7. Python Source Code Implementation (problem_100_smart_campus_simulation.py)

```
"""
Case Study 10: Simple Smart Campus Menu
================================================================
Syllabus Concepts Used:
- Variables, Data Types (int, float, str, bool)
- Relational and Logical Operators
- Conditional Statements (if, elif, else)
- Nested Loops and While Loops (Menu-Driven Architecture)
- Strings (formatting, centering, banners, alignment)
- Pattern Generation (Grid visualization and terminal banners)
- Lists and Nested Lists (2D Lab Seat Grids, Marks Matrix, Attendance Logs)
- Tuples, Tuple Concatenation, and Tuple Unpacking

Description:
A small terminal program showing a few campus activities:
1. Student Academic Registry
2. Attendance Monitoring & Condonation/Shortage Alert Engine (< 75% threshold)
3. Internal Marks Evaluation & Grade Determination
4. Campus Event Registration & Duplicate Shield
5. Computer Lab Seating Grid & Resource Allocation Matrix (2D Nested List)
6. Campus Summary
"""

def initialize_campus_data():
    """
    Initializes simulated campus datasets.
    """
    # Student Registry: List of tuples (roll_no, name, branch, section)
    students = [
        ("26B91A0501", "Aarav Sharma",    "CSE", "A"),
        ("26B91A0502", "Bhavya Reddy",    "CSE", "A"),
        ("26B91A0503", "Chaitanya V.",    "CSE", "B"),
        ("26B91A0504", "Divya Sri",       "ECE", "A"),
        ("26B91A0505", "Eshwar Rao",      "ECE", "B"),
    ]

    # Attendance: List of tuples (roll_no, total_classes, attended_classes)
    attendance = [
        ("26B91A0501", 100, 92),
        ("26B91A0502", 100, 84),
        ("26B91A0503", 100, 95),
        ("26B91A0504", 100, 68),  # Shortage (<75%)
        ("26B91A0505", 100, 72),  # Shortage (<75%)
    ]

... [Complete 331 lines available in problem_100_smart_campus_simulation.py]
```

8 & 9. Test Verification & Terminal Execution Output

